SMARTCRICKET AI: AN EXPLAINABLE MULTIMODAL CRICKET SHOT ANALYSIS AND INTELLIGENT VIRTUAL COACHING SYSTEM

A PROJECT REPORT

Submitted by

PRABHU

(Register No.: [Insert Register Number])

in partial fulfilment of the requirements for the award of the degree of

MASTER OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

[INSERT COLLEGE NAME]

[Insert College Address]

[INSERT UNIVERSITY NAME]

[Month] 2027

[INSERT UNIVERSITY NAME]

BONAFIDE CERTIFICATE

Certified that this project report **"SMARTCRICKET AI: AN EXPLAINABLE MULTIMODAL CRICKET SHOT ANALYSIS AND INTELLIGENT VIRTUAL COACHING SYSTEM"** is the bonafide work of **PRABHU ([Register Number])** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

[Insert Name]
HEAD OF THE DEPARTMENT
Professor
Department of Computer Science and Engineering
[College Name, Address]

SIGNATURE

[Insert Guide Name]
SUPERVISOR
[Designation]
Department of Computer Science and Engineering
[College Name, Address]

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, **PRABHU**, hereby declare that the project report entitled **"SmartCricket AI: An Explainable Multimodal Cricket Shot Analysis and Intelligent Virtual Coaching System"**, submitted in partial fulfilment of the requirements for the award of the degree of Master of Engineering in Computer Science and Engineering, is a record of original work carried out by me under the guidance of **[Insert Guide Name]**, [Designation], Department of Computer Science and Engineering, [Insert College Name].

I further declare that this work has not been submitted, in full or in part, for the award of any other degree or diploma in this or any other university. The CricShot10k dataset, the CricShotNet design and all other prior work used in this project are acknowledged and cited in the references.

Place: [Insert Place] **PRABHU**

Date: [Register Number]

ACKNOWLEDGEMENT

I express my sincere gratitude to the Management of **[Insert College Name]** for providing the facilities and environment needed to carry out this project.

I thank **[Insert Principal Name]**, Principal, for constant encouragement and support throughout the course of study.

I am grateful to **[Insert HoD Name]**, Professor and Head, Department of Computer Science and Engineering, for the guidance and for the opportunity to take up this work.

I owe a deep debt of gratitude to my project guide, **[Insert Guide Name]**, [Designation], Department of Computer Science and Engineering, for valuable suggestions, patient reviews and continuous motivation at every stage of the project.

I thank the project coordinator **[Insert Coordinator Name]** and the review committee members for their constructive comments during the Phase I and Phase II reviews.

I acknowledge M. K. Dihan, Abdullah, Amina and S. Ahmed, the authors of CricShot10k, for releasing the dataset and code under an open licence, which made this work possible.

Finally, I thank my family and friends for their encouragement and understanding throughout this work.

PRABHU

ABSTRACT

Cricket coaching and broadcast analysis still depend on people watching every ball and tagging each batting shot by hand. The work is slow, it does not scale across leagues and formats, and two annotators often label the same shot differently. Automatic recognition of batting shots from video is therefore a useful but difficult computer vision problem: shots are short, visually similar and filmed under changing camera angles, lighting and jersey colours.

This project, **SmartCricket AI**, designs and implements an explainable, multimodal system for cricket shot analysis with an intelligent virtual coach. It builds on the CricShot10k dataset of 10,086 one-second broadcast clips in 15 shot classes and its CricShotNet model (EfficientNetV2-S with a 128-unit GRU), which reports 89% top-1 accuracy. The system offers two modes. In *image mode*, a photograph of a batter is passed to MediaPipe Pose, and the 33 detected landmarks are turned into a posture report of joint angles and visibility scores. In *video mode*, a clip is sampled to 15 frames and classified by five models trained on the same split: video-only, pose-only and fused video + pose, with 2D or 3D pose features.

Each prediction carries a temperature-scaled confidence, a top-3 list, Grad-CAM heatmaps, an attention-based key-frame timeline and a Dynamic Time Warping comparison of joint-angle sequences against a reference clip. A large language model, given only the stored results, acts as a virtual coach that explains the outcome, separates measured facts from general tips. An adaptive training coach plans daily drills, and a live mirror counts repetitions on the device with a voice coach and a model player's ghost skeleton. The platform uses HTML5, CSS3 and JavaScript, FastAPI, PyTorch and PostgreSQL.

On 1,289 test clips, fusing video with 2D pose raised top-1 accuracy from 56.2% to 63.9% and macro-F1 from 55.3% to 62.2%, a statistically significant gain (McNemar $p < 10^{-8}$) that was largest for front-on camera views. 3D pose features did not improve recognition, but showed that 2D joint angles differ from 3D by 16 to 25 degrees depending on the view. Temperature scaling lowered the calibration error of all five models.

Keywords: Cricket shot classification, video action recognition, EfficientNetV2, GRU, MediaPipe Pose, Grad-CAM, temperature scaling, Dynamic Time Warping, large language model, FastAPI

TABLE OF CONTENTS

DECLARATION . . . . . **iii**

ACKNOWLEDGEMENT . . . . . **iv**

ABSTRACT . . . . . **v**

LIST OF FIGURES . . . . . **xi**

LIST OF TABLES . . . . . **xiii**

LIST OF ABBREVIATIONS . . . . . **xv**

1 INTRODUCTION **1**

1.1 Background 1

1.2 A Short Primer on Batting Strokes 2

1.3 Motivation 3

1.4 Problem Statement 3

1.5 Objectives 4

1.6 Scope of the Project 4

1.7 Organisation of the Report 5

2 LITERATURE SURVEY **6**

2.1 Recognising Shots from Single Images 6

2.2 Recognising Shots from Video 6

2.2.1 KUCricShot 6

2.2.2 CricShotClassify 7

2.2.3 Pose-landmark classification 7

2.2.4 CricShot10k and CricShotNet 7

2.3 Estimating Human Pose 9

2.4 Explaining and Trusting a Model's Output 9

2.5 Comparison of Related Work 10

2.6 Gaps Identified 10

3 SYSTEM ANALYSIS 12

3.1 Existing System 12

3.1.1 Manual shot tagging 12

3.1.2 CricShotNet pipeline 12

3.1.3 What the input looks like 13

3.1.4 Training and reported results 14

3.2 Disadvantages of the Existing System 14

3.3 Proposed System 15

3.3.1 Image mode 15

3.3.2 Video mode 15

3.3.3 AI virtual coach and platform 16

3.3.4 Key innovations 16

3.4 Feasibility Study 17

3.4.1 Technical feasibility 17

3.4.2 Economic feasibility 17

3.4.3 Operational feasibility 17

4 REQUIREMENTS SPECIFICATION 18

4.1 User Classes 18

4.2 Functional Requirements 18

4.3 Non-Functional Requirements 21

4.4 Hardware Requirements 22

4.5 Software Requirements 22

4.6 Legal, Ethical and Licensing Considerations 22

4.6.1 Dataset and image licences 23

4.6.2 Originality of the report 23

4.6.3 Privacy of users 23

4.6.4 Responsible use of AI feedback 23

5 SYSTEM DESIGN 24

5.1 System Architecture 24

5.2 Module Design 25

5.2.1 M2: Image analysis mode 26

5.2.2 M3: Video upload and preprocessing . . . 26
5.2.3 M4: Cricket shot classification . . . 26
5.2.4 M5: Pose estimation and movement comparison . . . 27
5.2.5 M6: Explainable AI . . . 27
5.2.6 M7: AI virtual coach . . . 28
5.3 Data Flow Diagrams . . . 28
5.3.1 Level 0 DFD . . . 28
5.3.2 Level 1 DFD . . . 28
5.4 UML Diagrams . . . 29
5.4.1 Use case diagram . . . 29
5.4.2 Class diagram . . . 30
5.4.3 Sequence diagram . . . 30
5.5 Database Design . . . 31
5.6 API Design . . . 32
5.7 User Interface Design . . . 33
5.8 Deployment Design . . . 33

6 IMPLEMENTATION . . . 34

6.1 Dataset Preparation . . . 34
6.2 Video Preprocessing (M3) . . . 35
6.3 Shot Classification Models (M4) . . . 35
6.3.1 Video-only model with attention . . . 35
6.3.2 Pose-only and fused models . . . 36
6.3.3 Training procedure . . . 36
6.3.4 Temperature scaling . . . 37
6.4 Pose Estimation and Joint Angles (M2, M5) . . . 37
6.5 Posture Module Output on Real Photographs . . . 38
6.6 Movement Analysis of Clips (M5) . . . 39
6.7 Reference Comparison with DTW (M5) . . . 40
6.8 Explainability with Grad-CAM (M6) . . . 40
6.9 Back-End API (M1, M8) . . . 40
6.9.1 Role-based access control . . . 40

6.10 AI Virtual Coach (M7) . . . 41
6.11 Adaptive Training Coach (M9) . . . 41
6.11.1 Choosing the drills . . . 41
6.11.2 Adapting the difficulty . . . 42
6.11.3 Levels, experience points and streaks . . . 42
6.11.4 Weekly report . . . 43
6.12 Real-World Input and Live Training . . . 43
6.12.1 Input quality check . . . 43
6.12.2 Following the batter . . . 44
6.12.3 Steadier angles and 3D pose . . . 44
6.12.4 Whole nets videos . . . 44
6.12.5 Live mirror and phone mode . . . 44
6.13 User Interface and Feature Walkthrough . . . 45
6.13.1 Landing page, sign-in and roles . . . 46
6.13.2 Player dashboard . . . 46
6.13.3 Photo posture report . . . 47
6.13.4 Video shot analysis and comparison . . . 48
6.13.5 AI Coach . . . 49
6.13.6 Daily training coach . . . 50
6.13.7 Live mirror . . . 51
6.13.8 Progress, history and reports . . . 52
6.13.9 Administrator console . . . 53

7 SYSTEM TESTING . . . 54

7.1 Testing Strategy . . . 54
7.2 Unit Testing . . . 55
7.3 Integration Testing . . . 56
7.4 System Testing . . . 56
7.5 Acceptance Testing . . . 58
7.6 Test Summary . . . 58

8 RESULTS AND DISCUSSION . . . 59

8.1 Evaluation Metrics . . . 59

8.2 Experiments 60
8.3 Classification Results (E1 to E3, E9) 61
8.4 Per-Class Analysis (E4) 63
8.5 Calibration Results (E5) 64
8.6 Pose Reliability (E6) 65
8.7 Robustness Across Camera Views and Sharpness (E10) 65
8.8 Explanations and Application Performance (E7, E8) 66
8.9 Discussion 67

9 CONCLUSION AND FUTURE ENHANCEMENTS 69

9.1 Conclusion 69
9.2 Limitations 70
9.3 Future Enhancements 70

REFERENCES 71

APPENDIX A: INSTALLATION GUIDE 75

APPENDIX B: IMAGE CREDITS AND LICENCES 76

LIST OF FIGURES

1.1 A batter moving onto the front foot to meet the ball 2

2.1 Automatic clip cutting: a clip begins only when the striker, ball and bowler are all detected 8

2.2 Preprocessing in CricShotNet: cropped frame (left) and the same frame with batter and bat segmented (right) 8

3.1 CricShotNet pipeline of the existing system (from [1]) 13

3.2 Sample frames from three CricShot10k clips, one clip per row 13

3.3 Overview of the proposed SmartCricket AI system 15

5.1 Layered architecture of SmartCricket AI 24

5.2 Classifier variants compared in experiments E1 to E3; E9 repeats the pose and fused models with 3D pose features 26

5.3 Landmarks and joint angles used by the posture report and comparison 27

5.4 Level 0 data flow diagram (context diagram) 28

5.5 Level 1 data flow diagram 29

5.6 Use case diagram 29

5.7 Class diagram 30

5.8 Sequence diagram for video analysis 31

5.9 Entity relationship diagram 31

6.1 Public landing page (left) and sign-in page (right) 46

6.2 Player dashboard with scoreboard, today's plan, batting profile, best photo, practice calendar and knee-angle trend 46

6.3 Photo posture report: quality score, body map with measured angles, detection quality, AI Coach and joint gauges 47

6.4 3D pose view with the 2D and 3D value of every joint angle and the gap between them 48

6.5 Video shot analysis: heatmap, attention timeline, predicted shot with calibrated confidence and top three, and the AI Coach 48

6.6 Comparison with a reference clip: DTW distance, per-joint differences, aligned frames and the angle curve of both strokes 49

6.7 AI Coach explaining a cover drive with measured facts and general tips (left), and starting the day on the Training page with a check-in and a question about the latest upload (right) 50

6.8 Level strip with level, experience points, streak and today's drill ring 50

6.9 Daily plan with the reason for each drill (left) and the question of the day with its explanation (right) 51

6.10 Live mirror tracking a batter at 59 frames per second (left) and "shadow the pro" with a 69% live match and a correction cue (right) 51

6.11 Live shot recognition (left) and phone mode with its QR code (right) 52

6.12 Weekly report with the coach's summary 52

6.13 History of analyses (left) and the one-click PDF report of a shot analysis (right) . 53

6.14 Administrator overview (left) and model results with the measured accuracy and calibration of the five models (right) 53

7.1 Levels of testing used in the project 54

8.1 Confusion matrix of the best model on the test split; rows are true shots, columns predicted shots 63

8.2 Macro-F1 of the five models by camera view 66

LIST OF TABLES

2.1 Comparison of cricket shot classification studies 10

3.1 Results reported for the existing system [1] 14

3.2 Innovations of the proposed system over [1] 16

3.3 Indicative cost estimate 17

4.1 User classes and their needs 18

4.2 Functional requirements 19

4.3 Non-functional requirements 21

4.4 Hardware requirements 22

4.5 Software requirements 22

5.1 Modules, objectives and requirements 25

5.2 Database tables and their main columns 32

5.3 REST API endpoints 33

6.1 Source code organisation 34

6.2 Training configuration 37

6.3 Joint angles computed from MediaPipe landmarks 38

6.4 Weak-spot rules used to choose drills 42

6.5 Levels and their requirements 43

6.6 Features of SmartCricket AI and their benefit 45

7.1 Unit test cases 55

7.2 Integration test cases 56

7.3 System test cases 57

8.1 Experiment plan 60

8.2 Classification results on the CricShot10k test split (1,289 clips) 61

8.3 McNemar exact test between models on the same test clips 62

8.4 Per-class results of the best model, E3 Video + pose (2D) 63

8.5 Calibration before and after temperature scaling 64

8.6 Landmark visibility across all sampled frames, and angle stability 65

8.7 Macro-F1 (%) by camera view and image sharpness 65

8.8 Measured response times on an Apple M4 laptop CPU (no GPU) 67
B.1 Sources of images used in this report 76

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Expansion
1	**AI**	Artificial Intelligence
2	**API**	Application Programming Interface
3	**ASGI**	Asynchronous Server Gateway Interface
4	**CC BY**	Creative Commons Attribution (licence)
5	**CNN**	Convolutional Neural Network
6	**CPU**	Central Processing Unit
7	**CSS**	Cascading Style Sheets
8	**DFD**	Data Flow Diagram
9	**DTW**	Dynamic Time Warping
10	**ECE**	Expected Calibration Error
11	**ER**	Entity Relationship
12	**FPS**	Frames Per Second
13	**GPU**	Graphics Processing Unit
14	**Grad-CAM**	Gradient-weighted Class Activation Mapping
15	**GRU**	Gated Recurrent Unit
16	**HTML**	HyperText Markup Language
17	**HTTP**	HyperText Transfer Protocol
18	**JSON**	JavaScript Object Notation
19	**JWT**	JSON Web Token
20	**LLM**	Large Language Model
21	**LRCN**	Long-term Recurrent Convolutional Network
22	**LSTM**	Long Short-Term Memory
23	**MIME**	Multipurpose Internet Mail Extensions
24	**MLP**	Multi-Layer Perceptron
25	**NLL**	Negative Log-Likelihood
26	**ORM**	Object Relational Mapper
27	**ReLU**	Rectified Linear Unit
28	**REST**	Representational State Transfer
29	**RNN**	Recurrent Neural Network
30	**SQL**	Structured Query Language
31	**SRS**	Software Requirements Specification
32	**ST-GCN**	Spatio-Temporal Graph Convolutional Network
33	**SVM**	Support Vector Machine
34	**TLS**	Transport Layer Security
35	**UI**	User Interface
36	**UML**	Unified Modeling Language
37	**XP**	Experience Points
38	**YOLO**	You Only Look Once

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Watch any cricket match on television and you will see the same short event repeated hundreds of times. The bowler runs in, the ball is delivered, and in roughly one second the batter decides on a stroke and plays it. That one-second window is where most of the interesting information in a match sits. A coach wants to know which strokes a young player leans on and which ones break down under pressure. An analyst preparing for an opponent wants to know where a batter scores and how they get out. A broadcaster wants to pull every cover drive of an innings into a highlight package before the next over begins.

All of these needs depend on a simple label: what shot was played? Surprisingly, that label is still mostly produced by a person sitting in front of a screen. The analyst pauses the video after each ball, decides on the shot and types it into a spreadsheet or a tagging tool. It is tiring work, it takes at least as long as the match itself, and it is less objective than it looks. A pull and a hook, or a square cut and a late cut, can look almost the same, and two experienced people will not always agree. At school and club level, where there is usually no analyst at all, a player's feedback is whatever the coach happened to notice from the side of the net.

Computer vision has matured to a point where much of this can be automated. Convolutional neural networks recognise people and objects reliably, recurrent networks can follow how a movement develops across frames, and pose estimators such as MediaPipe can locate the joints of a human body in a photograph in a few milliseconds on an ordinary laptop. These tools are already used to track players in football and basketball, to judge baseball swings and to classify tennis strokes [20]-[23].

What held cricket back was data. Earlier cricket shot datasets were small, covered only a handful of shots, or had problems with duplicated and mislabelled clips. In 2026, Dihan et al. published CricShot10k [1], a collection of 10,086 one-second broadcast clips covering 15 different batting shots, labelled independently by three annotators who agreed very strongly with each other. Alongside the data they released CricShotNet, a model that combines an EfficientNetV2-S image network with a small

recurrent layer and identifies the correct shot about 89% of the time. That work is the starting point of this project.

1.2 A SHORT PRIMER ON BATTING STROKES

Because this report is written for readers in computer science rather than cricket, it helps to describe what the system is trying to recognise. A batting stroke can be understood through three questions: which foot carries the weight, whether the bat comes down straight or across the line of the ball, and whether the ball is kept along the ground or hit in the air.

- **Front-foot strokes** are played by stepping towards the ball. The cover drive, straight drive, flick and forward defence belong to this group. Fig. 1.1 shows a batter moving onto the front foot to meet the ball.
- **Back-foot strokes** are played by moving back towards the stumps to a shorter ball. The square cut, late cut, pull and hook fall here.
- **Straight-bat versus cross-bat.** Drives and defensive strokes keep the bat vertical; cuts, pulls, hooks and sweeps swing the bat horizontally across the line of the ball.
- **Improvised strokes** such as the reverse sweep, scoop and upper cut are newer, mostly seen in limited-overs cricket, and deliberately break the classical rules.



Fig. 1.1 A batter moving onto the front foot to meet the ball

Photo: Acabashi, Wikimedia Commons, licensed under CC BY-SA 4.0.

Two practical points follow from this. First, many shots share the same starting stance and differ only in the last third of a second, so a model has to look at the sequence of frames rather than at a single picture. Second, the body tells a lot of the story: how far the front knee bends, where the head is relative to the front foot, and whether the shoulders open or stay side-on. This is why the project combines whole-frame video features with body-pose measurements.

1.3 MOTIVATION

The project grew out of three observations made while studying CricShot10k and earlier work.

A label alone does not help a player improve. If a system says "cover drive, 91%", a broadcaster has what they need, but a player does not. The player wants to know what their body did, what a good version of the stroke looks like, and what to practise. A coach, before trusting the system, wants to see that it is looking at the bat and the batter and not at the scoreboard. None of the published cricket shot systems provide that kind of evidence.

Research code is not something a coach can use. CricShot10k is released as data and training scripts. Someone who wants to analyse their own net session needs to install deep learning libraries, convert their video and run a script from the command line. There is no upload page, no history of past sessions and no report to share.

Body pose looks useful, but nobody has measured it on this dataset. Pose landmarks are not affected by jersey colour or lighting, which are two of the biggest nuisances in broadcast footage. On the other hand, a skeleton throws away the bat and the ball, and an earlier pose-only study on a smaller dataset [6] reported noticeably lower accuracy than video-based models. Whether pose adds anything on top of a strong video model is an open question, and it is the main research question of this project.

1.4 PROBLEM STATEMENT

The problem addressed in this project can be stated as follows. Given either a single photograph of a batter or a short video clip of one batting stroke, build a system that:

1. reports the batter's posture in measurable terms (joint angles with visibility) from a photograph, without guessing at joints it cannot see;
2. classifies the stroke in a video clip into one of the 15 CricShot10k classes, with a confidence value that reflects the real chance of being correct;

3. shows visual and temporal evidence for each classification;
4. compares the player's joint movement with a reference clip of the same stroke, even when the two were played at different speeds;
5. presents all of this through a web application and a conversational coach that explains the results in plain language without inventing facts.

In addition, the classifier must be evaluated fairly against the published baseline, and the effect of adding pose features must be measured on exactly the same data split.

1.5 OBJECTIVES

The objectives of the project are listed below. Each one is carried through to one or more modules in Chapters 5 and 6 and to one or more experiments in Chapter 8.

1. **Rebuild the baseline.** Re-implement the CricShotNet video model in PyTorch and train it on CricShot10k, so that every later comparison starts from a known, fair reference point.
2. **Add body pose.** Extract MediaPipe pose landmarks from photographs and from each sampled video frame, and turn them into joint angles and normalised coordinates.
3. **Compare five models.** Train video-only, pose-only (2D and 3D) and combined video + pose (2D and 3D) classifiers on the same split and compare them on accuracy, macro-F1, precision, recall, calibration and robustness to the camera view.
4. **Offer two analysis modes.** Provide photo analysis and video analysis in one web application with an HTML, CSS and JavaScript front end and a FastAPI back end.
5. **Explain and coach.** Produce Grad-CAM heatmaps, an attention-based key-frame timeline and a DTW comparison with a reference clip, and add an AI coach that explains stored results in everyday language and an adaptive training coach that plans daily practice from the player's own measurements and tracks progress over time.
6. **Make confidence honest.** Calibrate the classifier with temperature scaling and flag low-confidence clips as uncertain instead of guessing.

1.6 SCOPE OF THE PROJECT

Included: batting strokes by right- and left-handed batters in broadcast-style footage; the 15 shot classes of CricShot10k; single-person photographs for posture reports;

English-language coaching replies; deployment on a single server for academic evaluation.

Excluded: bowling and fielding; ball tracking and trajectory prediction; live streams; strokes outside the 15 classes such as the leave, leg glance and back-foot punch; medical, injury or biomechanical diagnosis; languages other than English.

One design decision deserves to be stated up front. Photo mode never predicts a shot type. A single still image cannot show how the bat is moving, and the literature shows that single-frame shot classification depends on a person choosing the right frame [1]. The photograph is therefore used only for what it can honestly support: a posture report.

1.7 ORGANISATION OF THE REPORT

Chapter 2 reviews related work in sports analytics, cricket video analysis, shot classification, pose estimation and explainable AI, and identifies the gaps this project fills. Chapter 3 studies the existing system, introduces the proposed system, and covers feasibility and project planning. Chapter 4 specifies the requirements and discusses legal and ethical considerations. Chapter 5 presents the system design. Chapter 6 explains the implementation of each module, including real output from the posture module. Chapter 7 describes testing. Chapter 8 sets out the evaluation and the results obtained so far. Chapter 9 concludes the report and suggests future work. References and appendices follow.

CHAPTER 2
LITERATURE SURVEY

A literature survey for this project has to cover more ground than a typical classification paper, because SmartCricket AI touches several fields at once: sports video analysis, action recognition, human pose estimation, model explainability, confidence calibration and language-based feedback. This chapter walks through each area, notes what each piece of work contributes and where it stops short, and ends with a comparison table and a list of the gaps that shaped the design of the project.

2.1 RECOGNISING SHOTS FROM SINGLE IMAGES

Several groups have tried to recognise batting shots from still images. Foysal et al. [13] trained a convolutional network named Shot-Net on frames of batters. Fernandes et al. [14] followed a similar route with a 2D CNN. Suryavanshi et al. [15] extracted CNN features and classified them with a support vector machine, while Dey and Biswas [16] applied a vision transformer to the same task. An older approach by Karmaker et al. [17] used motion vectors between frames, which works for a fixed camera but breaks down in broadcast footage where the camera pans to follow play.

The common weakness of the single-image approach is easy to see once the primer in Section 1.2 is kept in mind. Many shots start from the same stance and only become distinguishable as the bat comes through, so a single image either misses the decisive moment or requires a person to pick it out by hand. In other words, single-image methods move the hard part of the problem back to a human. This observation directly shaped our image mode: we keep photographs because club players often have nothing else, but we use them only for posture measurements and never for a shot label.

2.2 RECOGNISING SHOTS FROM VIDEO

Video-based work is more relevant, and also more limited in number. Four studies are discussed here in detail.

2.2.1 KUCricShot

Hoque et al. [11] collected about 1,300 short broadcast clips of four shot types and trained a Long-term Recurrent Convolutional Network [18], which passes each frame

through a CNN and then runs an LSTM across the sequence. They reached 73% accuracy. The dataset is a useful first step, but four classes is a small slice of real batting, and all clips come from Test matches with right-handed batters, so the model sees only white clothing and one batting direction. When the CricShot10k authors retrained it on coloured-clothing cricket, it did not generalise well [1].

2.2.2 CricShotClassify

Sen et al. [12] built a larger set of about 1,900 clips over ten shots and reported 93% accuracy with a VGG16 network followed by a GRU. On paper this is the highest accuracy in the area. Looking more closely, however, some clips are several seconds long, which means the network sees much more than the stroke itself. A share of the clips are mirrored copies of others, which inflates the dataset size and, if copies end up on both sides of the train/test split, can also inflate the accuracy [1]. The lesson for our project is practical: augmentation such as flipping must be applied only after the split, and the split must be checked for overlapping source videos.

2.2.3 Pose-landmark classification

Rao et al. [6] took a different route. They first detected the batter, then used MediaPipe to extract body landmarks and classified the resulting skeleton sequences, on a set of about 1,900 clips and six shots drawn from Bhat et al. [27]. They reported 80% accuracy over all classes and higher figures once some classes were removed. Their work shows that pose alone carries a lot of information about a stroke. It also shows the limit of pose alone: the skeleton does not include the bat, the ball or the background, so visually similar strokes become harder to separate. This raised the question we test in experiment E3: rather than replacing video with pose, does adding pose to video help?

2.2.4 CricShot10k and CricShotNet

The CricShot10k work by Dihan et al. [1] is the foundation of this project, so it is described in more detail. The authors started from 536 highlight videos totalling about 107 hours. Instead of cutting clips by hand, they used object detectors to find moments where the batter, the ball and the bowler were all on screen and cut a one-second clip at each such moment. This produced 14,369 candidate clips, of which 10,086 were kept after three annotators labelled them into 15 shot classes. All three annotators agreed on 9,218 clips, and their overall agreement, measured with Fleiss' kappa [36], was 0.9376, which is very high. Fig. 2.1 shows how the automatic cutting decides when a clip should start.

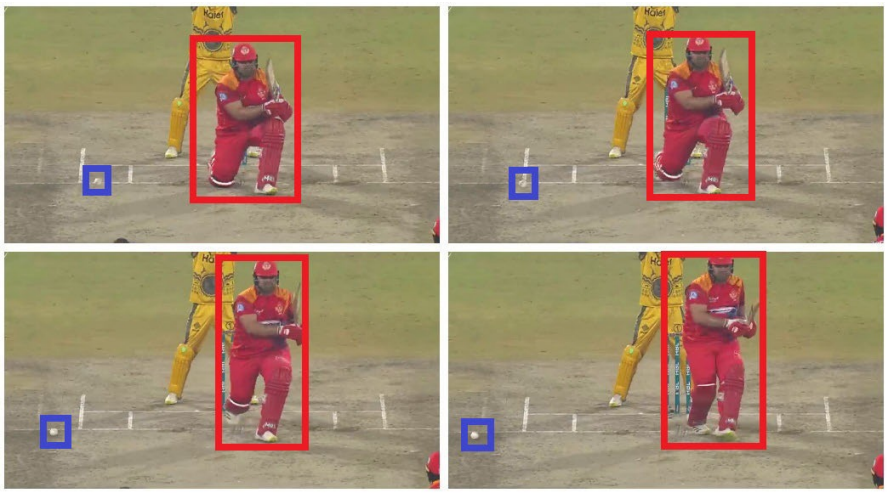


Fig. 2.1 Automatic clip cutting: a clip begins only when the striker, ball and bowler are all detected

Source: Dihan et al. [1], Fig. 4, licensed under CC BY 4.0.

For classification, the authors tried 15 image backbones and 9 recurrent variants and found that EfficientNetV2-S followed by a 128-unit GRU worked best. Two preprocessing steps added a few points of accuracy: cropping each frame around the batter, and tinting the batter and bat using a segmentation model so that the network pays less attention to clothing colour (Fig. 2.2). Their best model was right 89% of the time on the first guess and 99% of the time within its top three guesses. The pull and the hook were the pair most often confused, which matches the intuition from Section 1.2, since both are cross-bat strokes to short balls. A lighter MobileNetV3 version reached 84%.



Fig. 2.2 Preprocessing in CricShotNet: cropped frame (left) and the same frame with batter and bat segmented (right)

Source: Dihan et al. [1], Figs. 11 and 12, licensed under CC BY 4.0.

What CricShotNet does not do is equally important for this project. It returns a label and a probability, with no indication of what in the frames supported the decision and

no check on whether the probability is trustworthy. It was evaluated on broadcast footage only. And it is a research artefact rather than a tool a coach or player can use.

2.3 ESTIMATING HUMAN POSE

OpenPose [33] made real-time multi-person pose estimation widely available by predicting body parts and the connections between them. BlazePose [19], which powers MediaPipe Pose [43], focuses on one person at a time and returns 33 landmarks, each with a visibility score, fast enough to run on a phone. Graph-based models such as ST-GCN [34] treat a skeleton sequence as a graph that changes over time and are strong at recognising actions from skeletons alone.

MediaPipe is the right fit for this project for three reasons. It runs on a CPU, so the photo mode needs no GPU. It gives a visibility score for every landmark, which lets us refuse to report an angle for a joint that is hidden behind the body or the bat. And it offers an image mode and a video mode with the same landmark definitions, so a posture report from a photo and an angle sequence from a clip are directly comparable.

To compare a player's movement with a reference movement, the two sequences must first be lined up in time, because no two players swing at exactly the same speed. Dynamic Time Warping [10], originally developed for matching spoken words, finds the cheapest way to stretch and compress one sequence to match another, and it remains a standard tool for comparing gestures and motions.

2.4 EXPLAINING AND TRUSTING A MODEL'S OUTPUT

A prediction that cannot be checked is hard to trust. Grad-CAM [8] is a widely used way to see which parts of an image pushed a network towards a particular class. It looks at how strongly the class score depends on each feature map in the last convolutional layer, combines the maps accordingly, and overlays the result on the image as a heatmap. It needs no change to the network, so it can be added to an EfficientNet-based classifier after training.

Confidence is a second kind of trust. Guo et al. [9] showed that modern deep networks tend to be over-confident: when they say 90%, they are often right noticeably less often than that. Their remedy, temperature scaling, divides the network's raw scores by a single number learned on validation data. It does not change which class is predicted, only how confident the prediction looks, and it usually brings stated confidence much closer to real accuracy.

Finally, if a language model is going to talk to the player, it must not make things up. Retrieval-augmented generation [35] reduces this risk by giving the model the facts it is allowed to use instead of relying on what it has memorised. Our coach applies the same principle in a narrow form: the only facts it receives are the stored numbers for the current analysis.

2.5 COMPARISON OF RELATED WORK

Table 2.1 brings the most relevant shot classification studies together. Each accuracy figure comes from a different dataset with a different number of classes, so the numbers should not be compared directly; the table is meant to show coverage and limitations rather than to rank the methods.

Table 2.1 Comparison of cricket shot classification studies

Study	Data	Method	Accuracy	Main limitation
Foysal et al. 2019 [13]	Still images	CNN (Shot-Net)	-	One frame cannot show bat movement
Fernandes et al. 2023 [14]	Still images	2D CNN	-	One frame; few classes
Dey and Biswas 2024 [16]	Still images	Vision transformer	-	Key frame chosen by a person
Hoque et al. 2023 [11]	about 1,300 clips, 4 shots	LRCN	73%	Test cricket only, right-handers only
Sen et al. 2021 [12]	about 1,900 clips, 10 shots	VGG16 + GRU	93%	Long clips, mirrored copies counted
Rao et al. 2025 [6]	about 1,900 clips, 6 shots	MediaPipe pose	80%	No bat, ball or context in skeleton
Dihan et al. 2026 [1]	10,086 clips, 15 shots	EfficientNetV2-S + GRU	89%	No explanation, no calibration, no app

2.6 GAPS IDENTIFIED

Reading the work above side by side, six gaps stand out. Each one maps to a feature of SmartCricket AI.

1. **No explanations.** None of the cricket shot systems shows why it made a prediction. SmartCricket AI adds Grad-CAM heatmaps and a key-frame timeline.
2. **Untested value of pose.** Pose has been used alone, but not added to a strong video model on a large dataset. Experiment E3 tests exactly this.
3. **Untrustworthy confidence.** Reported probabilities are uncalibrated. SmartCricket AI applies temperature scaling and flags uncertain results.

4. **No comparison with good technique.** No system lines up a player's movement with a reference performance. SmartCricket AI uses DTW on joint angles.
5. **No usable product.** All prior work stops at a dataset and a model. SmartCricket AI is a working web application with history, reports and a coach.
6. **Misuse of single images.** Image-based methods try to read a shot from one frame. SmartCricket AI uses a photo only for what it can support: a posture report.

CHAPTER 3
SYSTEM ANALYSIS

System analysis studies the current way of solving the problem, identifies its weaknesses and defines how the proposed system will improve on it. This chapter describes the existing system, which is the CricShotNet pipeline published with the CricShot10k dataset [1], lists its disadvantages for the goals of this project, introduces the proposed SmartCricket AI system and its advantages, and closes with a feasibility study.

3.1 EXISTING SYSTEM

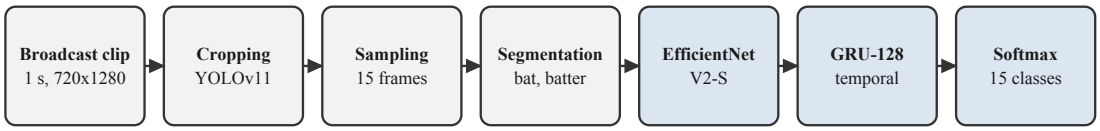
3.1.1 Manual shot tagging

In practice the existing system for most teams is manual. An analyst watches each delivery and records the shot in a spreadsheet or a commercial video-tagging tool. The analyst may also clip the delivery and attach notes. This process takes roughly as long as the footage itself, often longer, and depends on the experience of the person tagging. At academy level, coaches observe players live and give verbal feedback, with no stored record that a player can review later.

3.1.2 CricShotNet pipeline

The automated state of the art for cricket shot classification is CricShotNet [1]. Its pipeline, shown in Fig. 3.1, works as follows.

1. A one-second broadcast clip at 720 x 1280 pixels is taken as input.
2. YOLOv11 [29] detects the batter and the bat, and the frames are cropped around them.
3. Fifteen frames are sampled evenly from the clip and resized to 224 x 224 pixels.
4. A segmentation step tints the batter and the bat so that jersey colour and lighting matter less.
5. EfficientNetV2-S [3], pre-trained on ImageNet, extracts a feature vector from each frame.
6. A GRU with 128 units processes the 15 feature vectors in order and follows the swing over time.
7. A dense layer with softmax outputs a probability for each of the 15 shot classes.



Reported: top-1 89%, top-2 97%, top-3 99% on the CricShot10k test split [1]

Cropping + segmentation 89.09% vs cropping alone 85.76% [1]

Fig. 3.1 CricShotNet pipeline of the existing system (from [1])

3.1.3 What the input looks like

Fig. 3.2 shows real frames from three CricShot10k clips. Each row is one clip, with frames taken from the beginning, middle and end of the one-second window. Three things are worth noticing. The batter occupies only a small part of the wide broadcast frame, which is why the base paper crops around the batter before classification. The camera angle, pitch colour and clothing change from clip to clip. And most of the visible change between frames is in the batter's arms and bat, while the rest of the scene stays almost still.



Fig. 3.2 Sample frames from three CricShot10k clips, one clip per row

Source: Dihan et al. [1], Fig. 5, licensed under CC BY 4.0.

3.1.4 Training and reported results

The authors chose this configuration after comparing 15 CNN backbones and 9 recurrent variants. Horizontal flipping is applied to training clips only, to avoid leakage between splits, and training uses the Adam optimiser [38] with a learning rate of 10^{-4} and early stopping. The reported accuracies are summarised in Table 3.1.

Table 3.1 Results reported for the existing system [1]

Measure	Value
Top-1 accuracy, full CricShotNet	89%
Top-2 accuracy	97%
Top-3 accuracy	99%
Cropping + segmentation	89.09%
Cropping only	85.76%
MobileNetV3 variant, top-1	84%
Most confused pair	Pull vs hook (13%)
Inter-annotator agreement (Fleiss' kappa)	0.9376

3.2 DISADVANTAGES OF THE EXISTING SYSTEM

The points below are gaps with respect to the goals of this project. They are not failures shown in the experiments of [1]; where the paper itself supports a point, it is cited.

- **Fixed set of 15 shots.** The leave, leg glance and back-foot punch are not covered; the authors list them as future work [1].
- **Depends on picture conditions.** Camera angle, lighting, jersey colour and occlusion vary across clips [1]. Phone or academy video, which is filmed very differently from broadcast footage, has not been tested.
- **No explanation.** The output is a label and a probability. Nothing shows which part of the frame or which moment of the swing drove the answer, so a coach cannot check whether the model is right for the right reason.
- **Uncalibrated confidence.** The softmax probability is used directly. Deep networks are often over-confident [9], so a reported 0.9 may not mean that the prediction is right nine times out of ten.
- **No application or feedback.** The work is released as data and research code. There is no upload interface, no stored history, no comparison with a model player and no coaching feedback.

- **Pose is unproven here.** Pose alone loses texture and context [1], [6]; whether it helps when combined with the video model has not been tested, and it adds compute that must be justified.
- **No support for photographs.** Many club players only have a photograph of their stance or follow-through, which the existing system cannot use.

3.3 PROPOSED SYSTEM

SmartCricket AI keeps the strengths of CricShotNet and adds the missing parts around it. It offers two ways to analyse a batter, shown in Fig. 3.3, and shares one platform, one database and one AI coach between them.

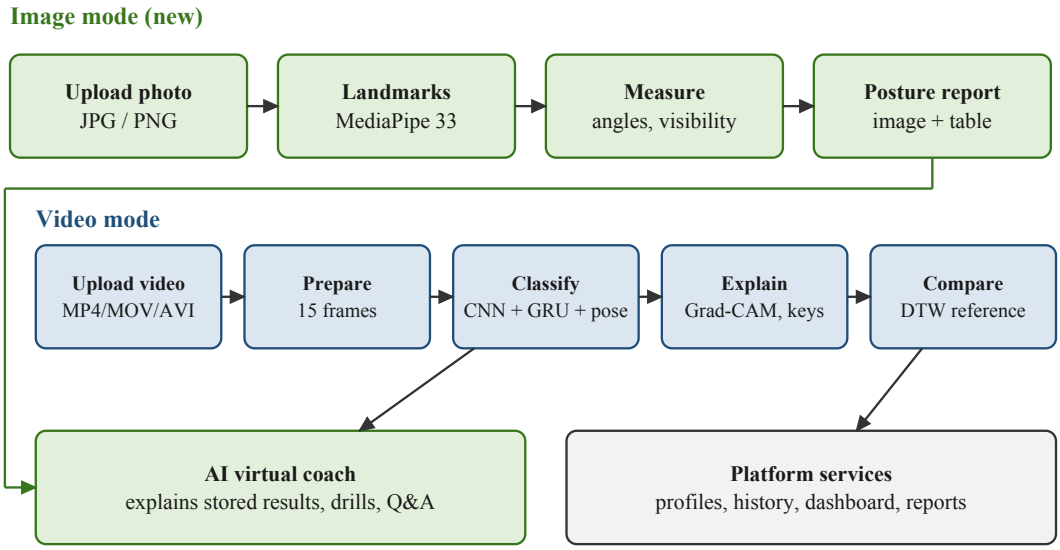


Fig. 3.3 Overview of the proposed SmartCricket AI system

3.3.1 Image mode

The user uploads a JPG or PNG photograph of a batter. MediaPipe Pose Landmarker [43] in image mode detects up to 33 body landmarks with visibility scores. The system computes front-knee, front-elbow, shoulder-line and hip angles from clearly visible points only and returns an annotated image drawn with OpenCV and a table of measurements. No shot type is predicted from a photograph, because one frame cannot show the movement of the bat [1].

3.3.2 Video mode

The user uploads a short MP4, MOV or AVI clip. OpenCV samples 15 evenly spaced frames and resizes them to 224 x 224. Several classifiers are available, of which the

three main ones are: the video-only model that re-implements CricShotNet, a pose-only model fed with the joint-angle sequence, and a fused video + pose model. The prediction is calibrated with temperature scaling, and clips below a confidence threshold are marked uncertain. Grad-CAM heatmaps show where the model looked in each frame, an attention layer over the GRU outputs ranks the frames into a key-frame timeline, and DTW aligns the player's joint-angle sequence with a reference clip of the same shot.

3.3.3 AI virtual coach and platform

A large language model, called from the server, receives only the stored results of the current session and explains them in plain words. Its reply separates "Measured" facts taken from the analysis from "General tip" coaching knowledge, suggests practice drills and answers follow-up questions. The platform adds player accounts, session history, a dashboard of shot mix and confidence trends, a model comparison page and downloadable CSV and HTML reports.

3.3.4 Key innovations

Table 3.2 lists what is new compared with the existing system.

Table 3.2 Innovations of the proposed system over [1]

No.	Feature	Description
1	Image analysis mode	Posture report from a single photograph: landmarks, angles and visibility.
2	Three-way comparison	Video-only, pose-only and video + pose trained and tested on the same split.
3	Grad-CAM heatmaps	Shows which parts of the bat and body led to the prediction in each frame.
4	Key-frame timeline	Attention weights show which of the 15 frames mattered most.
5	Reference comparison	Aligns the player's joint angles with a reference clip using DTW.
6	AI virtual coach	Explains results, suggests drills and answers follow-up questions, grounded in stored results.
7	Honest confidence	Temperature scaling so that a stated 80% means about 80%; unsure clips are flagged.
8	Player profiles	Session history, shot mix, trends and downloadable reports.

3.4 FEASIBILITY STUDY

A feasibility study checks whether the proposed system can be built and used with the available resources. Three aspects are considered.

3.4.1 Technical feasibility

All components use mature, open-source technology. PyTorch [41] provides pre-trained EfficientNetV2-S weights through torchvision. MediaPipe Pose runs on CPU in real time [19]. FastAPI [44] supports asynchronous file uploads, and PostgreSQL [45] is a stable relational database with JSON column support for joint-angle data. CricShot10k is publicly available with code [2]. Training the video model on 10,086 clips of 15 frames is within the capacity of a single consumer GPU with 8 to 12 GB of memory, or a free cloud notebook GPU, when frame features are cached. The project is therefore technically feasible.

3.4.2 Economic feasibility

The software stack is free and open source. The dataset is free to use for non-commercial research, which covers this academic project. Training can be done on an existing laptop GPU or on free and low-cost cloud notebooks. The only recurring cost is the language model API used by the coach, which is small because each request carries a short JSON summary rather than images or video. Table 3.3 gives an indicative cost estimate.

Table 3.3 Indicative cost estimate

Item	Estimated cost
Development software (Python, PyTorch, FastAPI, PostgreSQL, VS Code)	Free, open source
Dataset (CricShot10k)	Free for non-commercial research
GPU training	Existing hardware or free cloud notebook tier
LLM API for coach (evaluation period)	Low; pay per use
Hosting for demonstration	Local machine or college server

3.4.3 Operational feasibility

The application runs in any web browser with no installation, and uploading a file needs no technical knowledge. Results appear as a label, a percentage and pictures that the coach explains in plain English, and low-confidence results are flagged. The system is therefore operationally feasible.

CHAPTER 4
REQUIREMENTS SPECIFICATION

This chapter specifies what SmartCricket AI must do and the conditions under which it must do it. It follows the structure of a Software Requirements Specification (SRS): users of the system, functional requirements, non-functional requirements, hardware and software requirements, and a description of the selected technologies with the reasons for choosing them.

4.1 USER CLASSES

Table 4.1 User classes and their needs

User class	Description and needs
Player	Uploads photos and clips of their own batting, reads posture reports, movement analyses and shot predictions, compares with reference clips, chats with the coach, follows the daily training plan and tracks progress. Every new account is a player.
Administrator	Runs the platform rather than training on it: sees usage and training progress of all players (numbers only, never their media), installs the shot models and imports their test results, adds reference clips for each shot and reviews user accounts. Administrator rights are given only to email addresses listed in the server configuration.
LLM service	External system that receives a grounded prompt from the server and returns the coach's reply.

4.2 FUNCTIONAL REQUIREMENTS

Each functional requirement is identified as FR-n so that it can be traced to test cases in Chapter 7.

Table 4.2 Functional requirements

ID	Requirement	Description
FR-1	User registration	A new user registers with name, email and password. The email must be unique and the password at least 8 characters.
FR-2	Sign-in and sessions	A registered user signs in and receives a signed token. Protected endpoints reject missing or expired tokens with HTTP 401.
FR-3	Photo upload	The user uploads a JPG or PNG up to the configured size limit (default 10 MB). Other types are rejected with a clear message.
FR-4	Posture analysis	For a photo, the system detects up to 33 landmarks, reports the number visible and computes front-knee, front-elbow, shoulder-line and hip angles from points with visibility of at least 0.5.
FR-5	Annotated output	The system returns the photo with the skeleton drawn on it, low-visibility points shown in grey, and a table of measurements.
FR-6	Video upload	The user uploads an MP4, MOV or AVI up to the configured limit (default 50 MB). Unreadable or empty files are rejected.
FR-7	Frame preparation	The system samples 15 evenly spaced frames, repeating frames if the clip is shorter, and resizes them to 224 x 224.
FR-8	Shot classification	The system predicts one of 15 shot classes and returns the calibrated probability, the top-3 classes and the inference time.
FR-9	Uncertainty flag	If the calibrated top-1 probability is below the threshold (default 0.5), the result is marked uncertain.
FR-10	Model selection	The user can choose any installed model: video only, pose only (2D or 3D), or video fused with 2D or 3D pose; the best model is the default.
FR-11	Heatmaps	The system produces a Grad-CAM heatmap for each frame for the predicted class and overlays it on the frame.
FR-12	Key-frame timeline	The system shows the attention weight of each of the 15 frames.
FR-13	Reference comparison	The user selects a reference clip; the system aligns joint-angle sequences with DTW and returns per-joint differences and a total distance.
FR-14	AI coach	The user asks questions about a result; the coach replies using only stored results, labelling measured facts and general tips.
FR-15	History	Every analysis is stored and listed by date with its mode, model, result and thumbnail.

ID	Requirement	Description
FR-16	Dashboard	The dashboard shows clips analysed, average confidence, the most frequent shot, the shot mix and the confidence trend.
FR-17	Reports	The user downloads a CSV of their history and, with one click, a branded PDF report for any analysis.
FR-18	Administration	An administrator manages users, reference clips and the active model, views model evaluation results and sees usage and training progress for all players, without access to other players' media.
FR-19	Daily training plan	Each day the system builds a plan of three drills for the player, chosen from weak spots in their last five photos and recent clips, filled with general drills when there is not enough data.
FR-20	Check-in and safety	The coach asks the player for balls faced the day before, soreness (none, mild, needs rest) and how they feel. Mild soreness makes the plan easier; soreness that needs rest replaces the plan with a rest day and advice to see a doctor or physiotherapist if it continues.
FR-21	Levels, XP and streaks	The system awards experience points for analyses, finished drills, check-ins, quiz answers and practice days, keeps a daily streak, and moves the player through five levels with published requirements.
FR-22	Quiz and reflection	The system asks one batting question per day and, after a new upload, asks the player how the measured movement felt; answers are stored.
FR-23	Weekly report	The system summarises each week (days trained, drills finished, balls faced, soreness days, knee-angle change, quiz score), has the coach word the summary using only these numbers, and adapts the next week's difficulty to the completion rate.
FR-24	Movement analysis	For any clip, even before the shot models are trained, the system tracks the body in the 15 frames, draws the skeleton on each, reports joint angles over time, the range of each joint, and the frame with most body movement, ignoring camera zooms and cuts.
FR-25	Role separation	Player and administrator accounts see only their own pages. The server refuses player features (analysis, training, coaching, history, reports) to administrators and administrator features to players.

4.3 NON-FUNCTIONAL REQUIREMENTS

Table 4.3 Non-functional requirements

ID	Attribute	Requirement
NFR-1	Performance	Video analysis of a one-second clip completes within 3 s on a GPU server and within 15 s on CPU; photo analysis within 2 s.
NFR-2	Accuracy	The video-only model is compared with the 89% top-1 accuracy reported in [1]; because this project uses its own split and a frozen backbone, the gap and its causes are reported rather than a match being required.
NFR-3	Calibration	Temperature scaling reduces Expected Calibration Error on the test split compared with the uncalibrated model.
NFR-4	Security	Passwords are stored as salted hashes; tokens expire; the LLM API key is kept on the server only; uploads are validated by extension, MIME type and size.
NFR-5	Privacy	Each user sees only their own media and results. Only numeric results, never images, are sent to the LLM.
NFR-6	Usability	Every page works on screens from 360 px to 1920 px wide; errors are shown in plain language; the main task takes no more than three clicks.
NFR-7	Reliability	A failed analysis never leaves partial records; temporary files are removed; each failure is logged with its upload ID.
NFR-8	Maintainability	Code is organised in modules with type hints; configuration is held in environment variables; dependencies are pinned.
NFR-9	Portability	The back end runs on Linux, Windows and macOS with Python 3.10 or later.
NFR-10	Honesty	The coach never states numbers that are not in the stored result and says "not enough data" when confidence is low.

4.4 HARDWARE REQUIREMENTS

Table 4.4 Hardware requirements

Component	Minimum	Recommended
Processor	Intel Core i5 (8th gen) or equivalent	Intel Core i7 / AMD Ryzen 7
Memory (RAM)	8 GB	16 GB or more
GPU (training)	NVIDIA GPU with 6 GB VRAM	NVIDIA RTX 3060 (12 GB) or better
Storage	50 GB free	256 GB SSD
Display	1366 x 768	1920 x 1080
Client device	Any device with a modern browser	Laptop or tablet

4.5 SOFTWARE REQUIREMENTS

Table 4.5 Software requirements

Category	Software
Operating system	Ubuntu 22.04 LTS, Windows 10/11 or macOS 13+
Language	Python 3.10+, JavaScript (ES2020)
Deep learning	PyTorch 2.x, torchvision
Computer vision	OpenCV 4.x, MediaPipe 0.10+
Scientific computing	NumPy, scikit-learn
Web framework	FastAPI, Uvicorn, Pydantic
Database	PostgreSQL 15+, SQLAlchemy, psycopg
Front end	HTML5, CSS3, vanilla JavaScript, Chart.js
Language model	Any hosted LLM API with a chat endpoint
Development tools	VS Code, Git and GitHub, Python venv, Jupyter
Browser	Chrome, Edge, Firefox or Safari (current versions)

4.6 LEGAL, ETHICAL AND LICENSING CONSIDERATIONS

A system that handles photographs and videos of real people, uses a published dataset and calls an external AI service raises questions beyond the purely technical ones. They were considered from the start rather than at the end.

4.6.1 Dataset and image licences

Two different permissions apply to the base work. The CricShot10k article itself is published under the Creative Commons Attribution 4.0 licence, so its figures may be reused with credit; every figure in this report taken from the paper says so in its caption [1]. The dataset is different: its authors state that it is intended solely for non-commercial research, and the copyright of the original broadcast videos stays with their rights holders [1], [2]. This project uses the dataset only to train and evaluate models for academic research. No clips are redistributed, and the trained models and any demonstration are not offered commercially. A commercial version of SmartCricket AI would need its own licensed training footage. The photographs of batters used in Chapters 1 and 6 come from Wikimedia Commons under the Creative Commons Attribution-ShareAlike 4.0 licence, which also requires credit and requires that adapted versions, such as our annotated pose images, are shared under the same licence. A complete list of image sources is given in Appendix B. All diagrams, charts and wireframes not marked otherwise were created for this report.

4.6.2 Originality of the report

The text of this report was written for this project. Facts, figures and methods taken from other work are paraphrased in our own words and cited in IEEE style; no passage is reproduced from another document. Numbers reported by other authors are always marked with their citation and are never presented as results of this project.

4.6.3 Privacy of users

Photographs and videos uploaded by players are personal data. The design keeps them on the project server, links them to the uploading user only, and never sends them to the language model: the coach receives only numbers such as angles and probabilities. Users can delete an analysis, which removes the record and its files. For a real deployment, the privacy notice would also state how long files are kept, and uploads of minors would need consent from a parent or guardian.

4.6.4 Responsible use of AI feedback

Joint angles measured from a single camera are 2D estimates and depend on where the camera stands. The application and the coach therefore describe them as estimates, never as a judgement of correct or incorrect technique, and never give medical or injury advice. Low-confidence predictions are labelled as uncertain rather than presented as facts. These rules are enforced in code (Section 6.10) and checked in testing (ST-10).

CHAPTER 5
SYSTEM DESIGN

System design turns the requirements of Chapter 4 into a structure that can be implemented. This chapter presents the overall architecture, the design of each module, the data flow diagrams, the UML diagrams, the database design and the user interface design.

5.1 SYSTEM ARCHITECTURE

SmartCricket AI follows a layered client-server architecture, shown in Fig. 5.1. The browser talks only to the API layer. The API layer validates requests and passes media to the preprocessing layer, which prepares inputs for the two inference paths: the shot classifier and the pose analysis path. Their outputs are merged by the results service, stored in PostgreSQL and file storage, and passed to the AI coach on request.

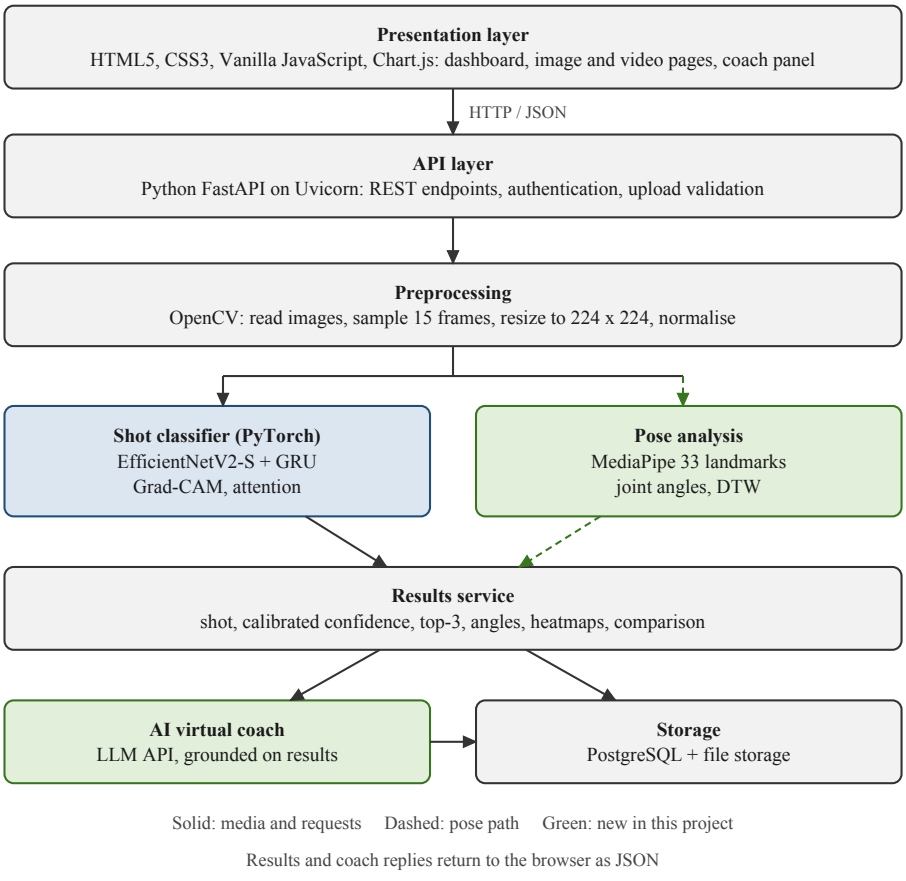


Fig. 5.1 Layered architecture of SmartCricket AI

The layers have the following responsibilities.

- **Presentation layer.** Static HTML, CSS and JavaScript pages for the dashboard, image mode, video mode, coach and history. Calls the REST API with Fetch and renders JSON results with Chart.js.
- **API layer.** FastAPI routers for authentication, image analysis, video analysis, references, coach, history and reports. Handles token checks, request validation and error responses.
- **Preprocessing layer.** OpenCV routines for decoding, sampling, resizing and normalising. The same code is used during training and inference so that the model sees identical inputs.
- **Inference layer.** The PyTorch classifier with Grad-CAM and attention, and the MediaPipe pose path with angle calculation and DTW.
- **Results and coach.** Merges outputs into one result object, applies calibration and the uncertainty flag, and builds grounded prompts for the LLM.
- **Storage.** PostgreSQL for structured records; the file system for uploads, heatmaps and reports.

5.2 MODULE DESIGN

The system is divided into nine modules. Table 5.1 maps each module to the objectives of Chapter 1 and the functional requirements of Chapter 4.

Table 5.1 Modules, objectives and requirements

No.	Module	Objective	Requirements
M1	User interface and authentication	4	FR-1, FR-2, FR-15, FR-16
M2	Image analysis mode	2, 4	FR-3, FR-4, FR-5
M3	Video upload and preprocessing	1, 4	FR-6, FR-7
M4	Cricket shot classification	1, 3, 6	FR-8, FR-9, FR-10
M5	Pose estimation and movement comparison	2, 5	FR-13
M6	Explainable AI: heatmaps and key frames	5	FR-11, FR-12
M7	AI virtual coach	5	FR-14
M8	Analytics, database and reporting	4	FR-16, FR-17, FR-18
M9	Adaptive training coach	4, 5	FR-19 to FR-23

5.2.1 M2: Image analysis mode

A validated photo is decoded with OpenCV, converted to RGB and passed to MediaPipe Pose Landmarker in image mode. For each landmark the module keeps the normalised coordinates and the visibility score. Angles are computed only from points whose visibility is at least 0.5; others are greyed out in the report, never guessed. The annotated image and the angle table are saved and linked to the analysis record.

5.2.2 M3: Video upload and preprocessing

The upload is checked for extension, MIME type, size and readability. OpenCV VideoCapture reads the frame count, 15 evenly spaced indices are chosen, frames are read, converted to RGB, resized to 224 x 224 and normalised with ImageNet mean and standard deviation. The result is a tensor of shape 15 x 3 x 224 x 224. Errors return HTTP 400, 413 or 422 with a clear message, and temporary files are deleted.

5.2.3 M4: Cricket shot classification

Five models are trained on the same split (Fig. 5.2). The video-only model follows the CricShotNet design: a frozen, ImageNet-trained EfficientNetV2-S produces a 1280-dimensional feature for each frame and a GRU with 128 units processes the sequence. An additive attention layer over the GRU outputs produces both a context vector for classification and the per-frame weights used by the key-frame timeline. The pose-only model feeds a 43-value per-frame vector of joint angles, normalised landmark coordinates and a frame-usable flag to a GRU with 64 units. The fused model concatenates the two sequence summaries and classifies them with a small multi-layer perceptron. All three end with a softmax over 15 classes, scaled by a learned temperature.

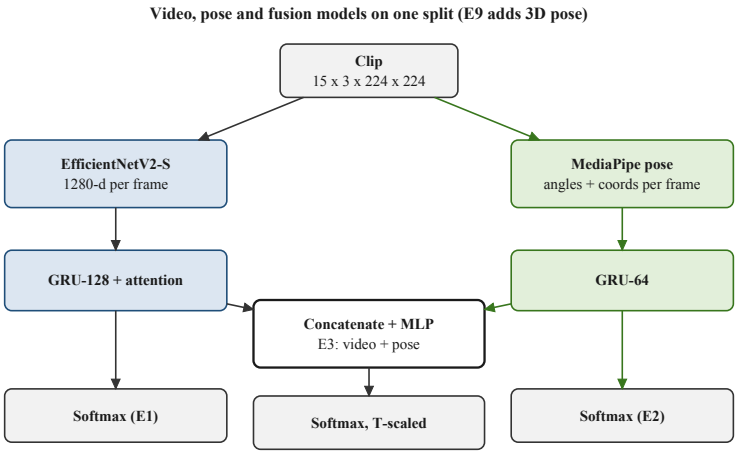


Fig. 5.2 Classifier variants compared in experiments E1 to E3; E9 repeats the pose and fused models with 3D pose features

5.2.4 M5: Pose estimation and movement comparison

MediaPipe in video mode returns landmarks for each sampled frame. For each frame the module computes the elbow, knee, shoulder and hip angles. The user chooses a reference clip of the same shot, stored with its own pre-computed angle sequence. DTW aligns the two multi-joint sequences even when the shots are played at different speeds, and the module returns the alignment path, the total distance and the mean difference per joint.

The same pose path also gives a movement analysis that needs no trained classifier. Joint angles are measured on every frame, each joint's range over the stroke is recorded, and the frame with the most body movement is found. Movement is measured relative to the batter's own hips and in units of shoulder width, so camera pans and zooms are not counted as movement, and a sudden change of body size is treated as a camera cut. This lets video mode give useful feedback from the first day, before the shot models are trained.

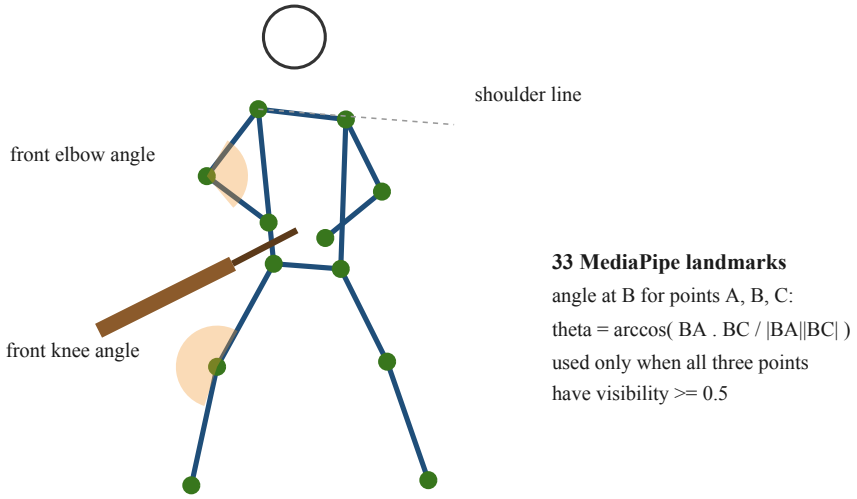


Fig. 5.3 Landmarks and joint angles used by the posture report and comparison

5.2.5 M6: Explainable AI

Grad-CAM uses the gradient of the predicted class score with respect to the feature maps of the last EfficientNet convolutional block for each frame. The weights are global averages of the gradients; the weighted sum of feature maps, passed through ReLU and upsampled, is blended onto the frame with an OpenCV colour map. The attention weights from M4 form the key-frame timeline. The interface shows heatmap, pose skeleton and timeline side by side.

The real heatmaps produced by the trained model are shown on the video result page in Section 6.13.

5.2.6 M7: AI virtual coach

The coach reads the stored analysis, never the raw media. A prompt builder combines fixed system rules, the analysis as JSON and the last six messages of the conversation. The rules require the model to label statements as "Measured" when they come from the analysis and "General tip" when they are coaching knowledge, to suggest drills as suggestions only, to answer "not enough data" when the result is marked uncertain, and to refuse medical or injury advice. A guard on the server checks that every number in the reply appears in the stored result.

5.3 DATA FLOW DIAGRAMS

5.3.1 Level 0 DFD

The context diagram (Fig. 5.4) treats SmartCricket AI as a single process exchanging data with three external entities: the player or coach, the administrator and the LLM service.

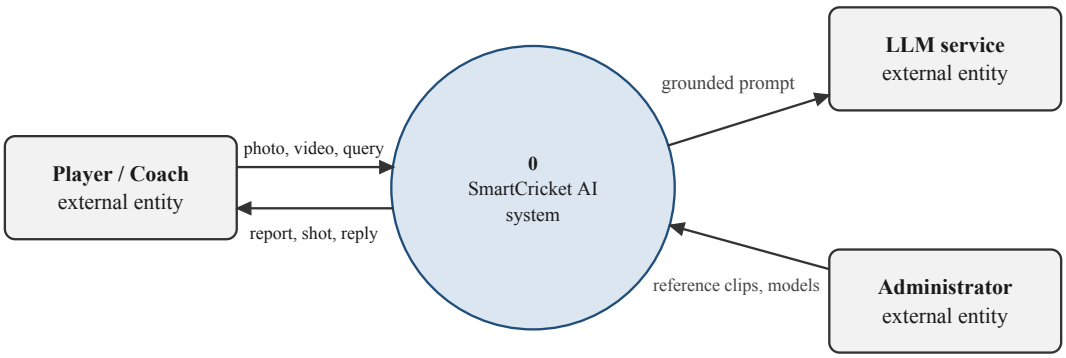


Fig. 5.4 Level 0 data flow diagram (context diagram)

5.3.2 Level 1 DFD

The level 1 diagram (Fig. 5.5) splits the system into five processes: authentication, upload validation, image analysis, video analysis, and coaching and reporting, with four data stores.

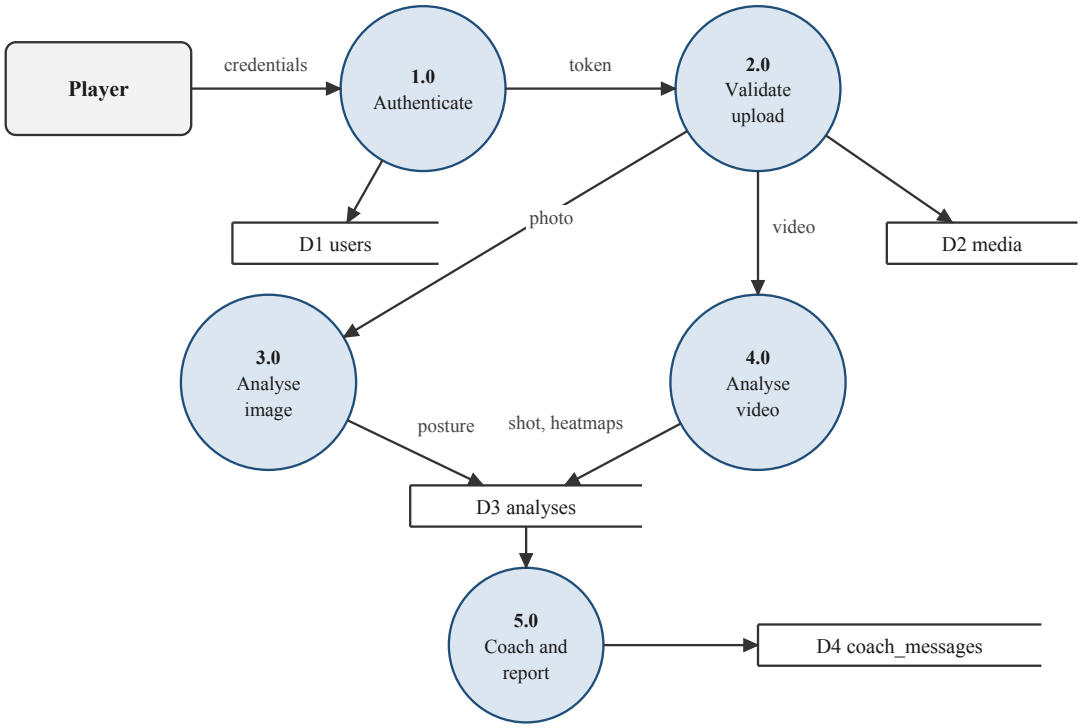


Fig. 5.5 Level 1 data flow diagram

5.4 UML DIAGRAMS

5.4.1 Use case diagram

Fig. 5.6 shows the actors and their use cases. The player or coach performs all analysis use cases; the administrator manages reference clips, users and models; the LLM service participates in the coach use case.

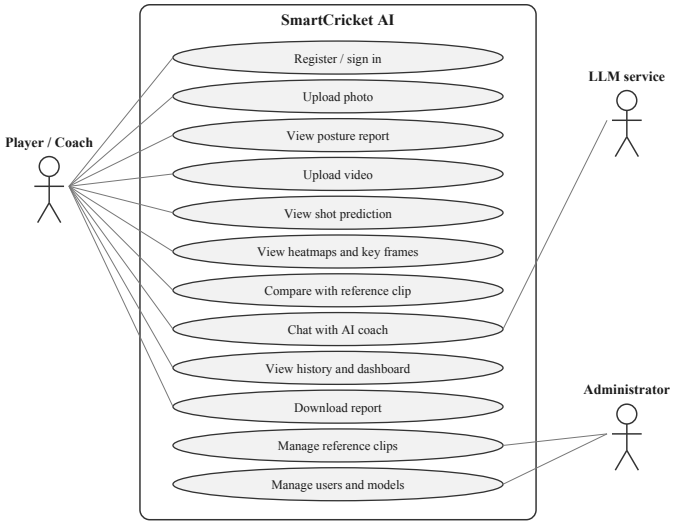


Fig. 5.6 Use case diagram

5.4.2 Class diagram

Fig. 5.7 shows the main classes. User, Media and Analysis are persistent entities. VideoPreprocessor, ShotClassifier and PoseEstimator implement the analysis pipeline, and CoachService reads stored analyses.

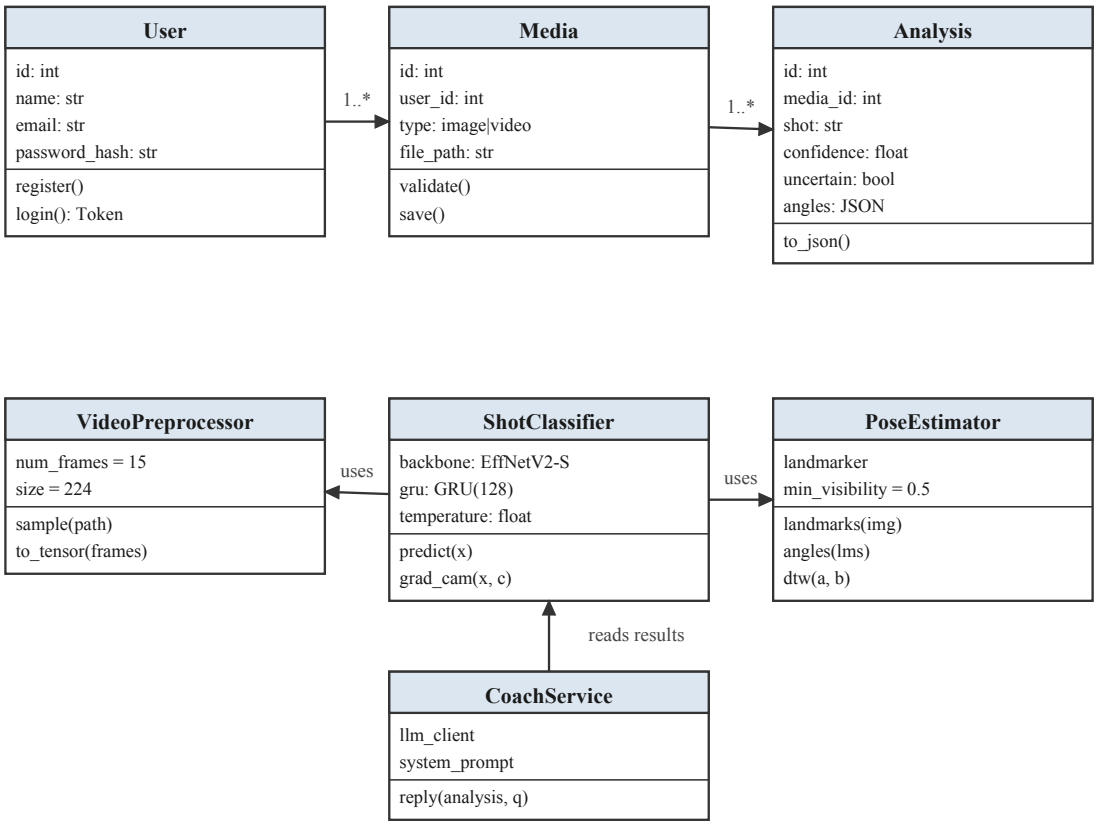


Fig. 5.7 Class diagram

5.4.3 Sequence diagram

Fig. 5.8 shows the sequence of messages for a video analysis request, from upload to the JSON response.

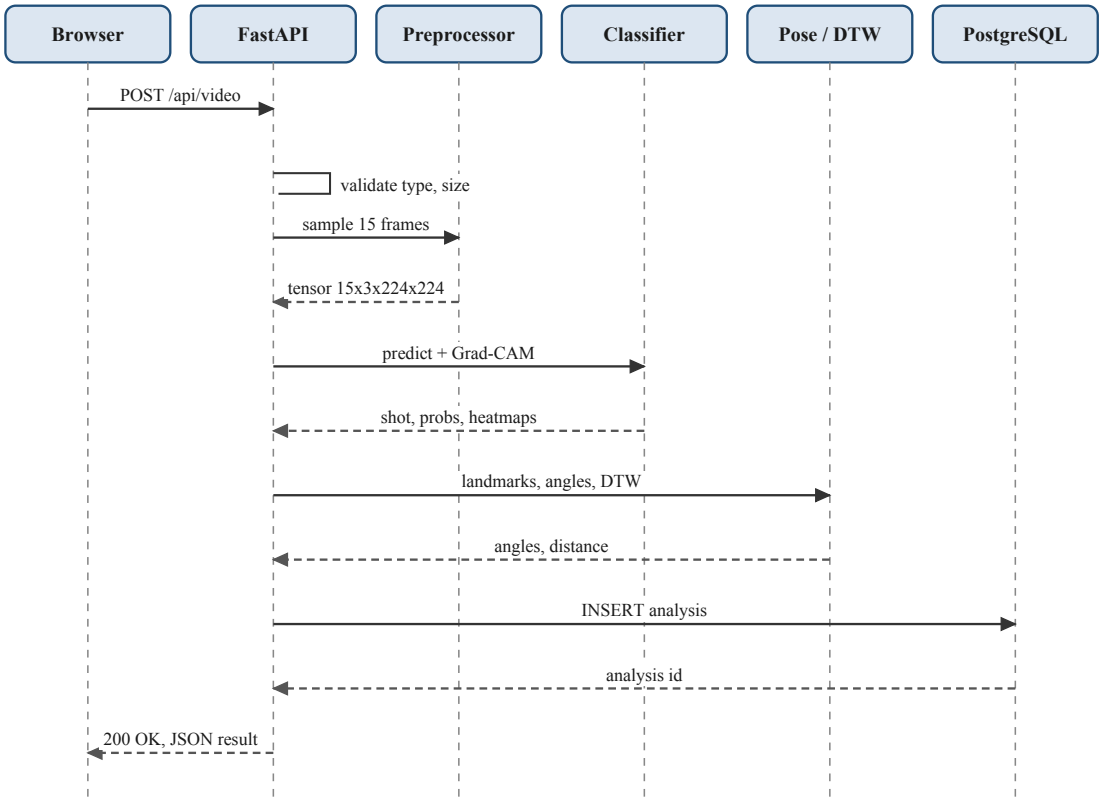


Fig. 5.8 Sequence diagram for video analysis

5.5 DATABASE DESIGN

The database has twelve tables in PostgreSQL. The six core tables and their relationships are shown in Fig. 5.9; Table 5.2 lists every table with its main columns. All tables use an integer primary key, every player-owned row references users.id and is deleted with its user, and JSON columns (JSONB in PostgreSQL) hold per-frame results that do not need separate querying.

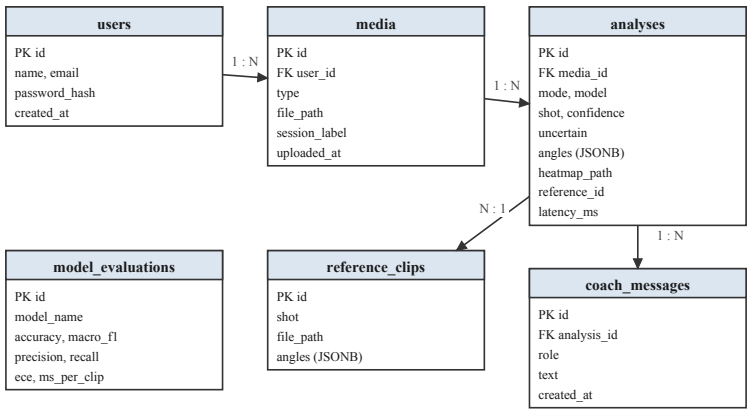


Fig. 5.9 Entity relationship diagram

Table 5.2 Database tables and their main columns

Table	Main columns	Purpose
users	name, email (unique), password_hash (bcrypt), role, created_at	Accounts; role is player or admin
media	user_id, type, file_path, original_name, session_label, uploaded_at	Uploaded photos and videos
analyses	media_id, mode, model, shot, confidence, uncertain, top3, angles, attention, summary (JSON), dtw, latency_ms	One result per analysis
coach_messages	analysis_id, role, text, created_at	Conversation about one result
reference_clips	shot, file_path, angles (JSON), added_by	Model-player clips for comparison
model_evaluations	model_name, accuracy, macro_f1, precision, recall, ece_before, ece_after	Imported test results
daily_logs	user_id, day (unique together), balls_faced, soreness, feeling	Morning check-in
plan_drills	user_id, day, code, title, dose, reason, difficulty, done	The daily drill plan
quiz_answers	user_id, day (unique together), question, choice, correct	Question of the day
reflections	user_id, analysis_id (unique), question, answer	Answers to the coach's question about an upload
weekly_reports	user_id, week_start (unique together), text, stats	Worded weekly summary
training_chats, live_sessions	user_id, role / text; title, duration_s, reps, hold_s, stats	Training chat and saved live sessions

5.6 API DESIGN

The back end exposes the REST endpoints in Table 5.3. All endpoints except registration and sign-in require a bearer token.

Table 5.3 REST API endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Create an account
POST	/api/auth/login	Sign in and receive a token
POST	/api/image/analyse	Upload a photo and get a posture report
POST	/api/video/analyse	Upload a clip and get shot, confidence, top-3, heatmaps
POST	/api/video/{id}/compare	Compare an analysis with a reference clip using DTW
POST	/api/coach/{analysis_id}	Send a question; receive the coach's reply
GET	/api/history	List the user's analyses
GET	/api/dashboard	Aggregated numbers and chart data
GET	/api/reports/{id}/pdf	Branded PDF report, downloaded directly
GET	/api/train/today	Today's plan, check-in, quiz and coach questions
POST	/api/train/checkin	Save the morning check-in and rebuild the plan if needed
POST	/api/train/chat	Free question to the training coach
GET	/api/train/progress	Levels, XP breakdown and eight weeks of statistics
POST	/api/train/week/summary	Coach-worded weekly summary
GET	/api/admin/overview	Usage and training progress of all players (administrator)

5.7 USER INTERFACE DESIGN

The interface follows four principles: one main action per page, results visible without scrolling on a laptop screen, plain language with units on every number, and the evidence (heatmap, timeline, skeleton) next to the claim it supports. The pages were designed around these principles and then built; the final screens are shown in Section 6.13.

5.8 DEPLOYMENT DESIGN

For evaluation, the system is deployed on a single application server. Nginx serves static files and terminates TLS, Uvicorn runs the FastAPI application with the PyTorch and MediaPipe models loaded once at start-up, PostgreSQL runs on the same machine or a separate one, and the coach calls the external LLM API over HTTPS.

CHAPTER 6
IMPLEMENTATION

This chapter describes how each module of Chapter 5 is implemented. For each module it gives the algorithm, the key equations and a short code listing taken from the project source. The full source is organised as shown in Table 6.1.

Table 6.1 Source code organisation

Path	Contents
backend/app/main.py	FastAPI application, routers, start-up model loading
backend/app/auth.py	Registration, password hashing, token issue and check
backend/app/preprocess.py	Upload validation, frame sampling, tensor preparation
backend/app/models/	ShotNet, PoseNet, FusionNet, temperature scaler
backend/app/pose.py	MediaPipe wrapper, joint angles, DTW
backend/app/explain.py	Grad-CAM and overlay drawing
backend/app/coach.py	Prompt builder, LLM client, number guard
backend/app/db.py	SQLAlchemy models and session
training/train.py	Training loop for E1 to E3, early stopping
training/evaluate.py	Metrics, confusion matrix, ECE, latency
frontend/	index.html, image.html, video.html, history.html, css/, js/

6.1 DATASET PREPARATION

CricShot10k is downloaded from the authors' repository [2]. The authors do not publish their train, validation and test lists, so this project makes its own stratified split of 70% training, 15% validation and 15% test with a fixed random seed, and saves it so that every model and every run uses exactly the same clips. Stratification keeps the share of each shot the same in all three parts. When the file names show which source video a clip came from, clips of one source are kept together in a single part (stratified group split), so that near-identical frames from the same match cannot appear in both training and test. Because the split differs from the authors', results are compared with [1] as a reference point rather than as a like-for-like benchmark (see the Discussion in Chapter 8).

Each clip is processed once and the results are cached on Google Drive in shards of 500 clips: the 1280-dimensional EfficientNetV2-S feature of each of the 15 frames, the same features for the mirrored clip, and the MediaPipe landmarks of the 15 frames. Training then reads only these small arrays, so an epoch takes seconds instead of minutes, and an interrupted Colab session can resume where it stopped.

Horizontal flipping is applied to training clips only, with probability 0.5, by choosing the cached features of the mirrored clip. When a clip is flipped, the left and right landmarks are swapped and x-coordinates are mirrored so that the pose features stay consistent with the image. Because frame features are computed once, augmentations that would change them at every epoch, such as colour jitter or random crops, are not used. The class imbalance ratio of 4.26 [1] is handled with class-weighted cross-entropy, with weights inversely proportional to class frequency.

6.2 VIDEO PREPROCESSING (M3)

The frame sampler chooses 15 evenly spaced indices across the clip. If a clip has fewer than 15 frames, indices repeat, as required by FR-7. The same function is used for training and inference.

Upload validation checks the file extension against an allow-list, the MIME type reported by the client, the size against the configured limit, and finally whether OpenCV can open the file. Each failure maps to a specific HTTP status: 400 for a wrong type, 413 for a file that is too large and 422 for an unreadable file.

6.3 SHOT CLASSIFICATION MODELS (M4)

6.3.1 Video-only model with attention

Each of the T = 15 frames x_t passes through the EfficientNetV2-S backbone with its classification head removed, giving a feature vector f_t of 1280 dimensions. The GRU [4] updates its hidden state h_t as follows.

$$z_t = sigma(W_z f_t + U_z h_{t-1}),\ r_t = sigma(W_r f_t + U_r h_{t-1}) \qquad (6.1)$$

$$h_t = (1 - z_t) h_{t-1} + z_t \tanh(W f_t + U (r_t h_{t-1})) \qquad (6.2)$$

Additive attention [40] scores each hidden state and forms a context vector c, which is passed to the classifier. The weights a_t are the key-frame timeline.

$$e_t = v^T \tanh(W_a h_t),\ a_t = \exp(e_t) / sum_k \exp(e_k),\ c = sum_t a_t h_t \qquad (6.3)$$

Listing: training/sc_common.py - video model

```
import torch, torch.nn as nn
from torchvision.models import efficientnet_v2_s, EfficientNet_V2_S_Weights

class ShotNet(nn.Module):
    def __init__(self, n_classes=15, hidden=128):
        super().__init__()
        cnn = efficientnet_v2_s(weights=EfficientNet_V2_S_Weights.IMAGENET1K_V1)
        self.features = cnn.features          # last conv block used by Grad-CAM
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.gru = nn.GRU(1280, hidden, batch_first=True)
        self.att = nn.Sequential(nn.Linear(hidden, 64), nn.Tanh(), nn.Linear(64, 1))
        self.head = nn.Sequential(nn.Dropout(0.3), nn.Linear(hidden, n_classes))

    def encode(self, x):                      # x: (B, T, 3, 224, 224)
        B, T = x.shape[:2]
        f = self.pool(self.features(x.flatten(0, 1))).flatten(1)
        h, _ = self.gru(f.view(B, T, -1))     # (B, T, hidden)
        a = torch.softmax(self.att(h).squeeze(-1), dim=1)
        return (a.unsqueeze(-1) * h).sum(1), a

    def forward(self, x):
        c, a = self.encode(x)
        return self.head(c), a
```

6.3.2 Pose-only and fused models

For the pose path, each frame gives a 43-dimensional vector: eight joint angles (left and right elbow, shoulder, hip and knee) scaled to the range 0 to 1, the x and y coordinates of 17 upper- and lower-body landmarks, centred on the hip midpoint and divided by the shoulder width so that the vector does not depend on where the batter stands or how large they appear, and one flag that is 1 when the frame is usable. A joint whose visibility is below 0.5 is set to zero instead of being guessed. A frame in which the shoulders or hips cannot be found has its flag at 0 and takes the values of the last usable frame. A GRU with 64 units summarises the sequence.

The fused model concatenates the 128-dimensional video context vector and the 64-dimensional pose summary and passes them through a two-layer perceptron (192 to 128 to 15) with dropout of 0.3. Late fusion was chosen over early fusion because it lets each stream keep its own temporal model and works directly on the cached features of both streams.

6.3.3 Training procedure

All five models are trained with the same settings so that differences come from the inputs, not the schedule (Table 6.2). The EfficientNetV2-S backbone keeps its ImageNet weights and stays frozen throughout; only the GRU, attention and classification layers are trained. This is a deliberate trade-off for the available hardware: fine-tuning the backbone on 10,000 clips of 15 frames needs far more GPU

time than a free Colab session gives, while frozen features can be computed once and reused for all runs. It is also the main expected reason for any accuracy gap to the base paper, which fine-tuned the network [1].

Table 6.2 Training configuration

Setting	Value
Optimiser	AdamW, learning rate 10^{-3} with 5% linear warm-up and cosine decay, weight decay 10^{-4}
Backbone	EfficientNetV2-S with ImageNet weights, frozen (features cached once)
Batch size	64 clips for all models
Loss	Class-weighted cross-entropy, label smoothing 0.05
Epochs	Up to 40, early stopping on validation macro-F1 with patience 6
Precision	Mixed precision (float16) for feature extraction on GPU
Augmentation	Horizontal flip of training clips only, with mirrored landmarks
Random seeds	Fixed for Python, NumPy and PyTorch; three runs per model

6.3.4 Temperature scaling

After training, a single temperature T is learned on the validation set by minimising negative log-likelihood with the model weights frozen [9]. At inference time the calibrated probabilities are as follows; T greater than 1 softens over-confident outputs without changing the predicted class.

$$p_i = exp(z_i / T) / sum_j \, exp(z_j / T) \qquad (6.4)$$

Calibration is measured with the Expected Calibration Error over M = 15 equal-width confidence bins B_m, where n is the number of test clips.

$$ECE = sum_m \, (|B_m| / n) \, |acc(B_m) - conf(B_m)| \qquad (6.5)$$

6.4 POSE ESTIMATION AND JOINT ANGLES (M2, M5)

MediaPipe Pose Landmarker is created once at start-up, in image mode for photos and in video mode for clips. For three landmarks A, B and C, the angle at B is computed from the 2D vectors BA and BC.

$$theta_B = arccos((BA \, . \, BC) / (|BA| \, |BC|)) \qquad (6.6)$$

An angle is reported only when all three landmarks have visibility of at least 0.5 (FR-4). Table 6.3 lists the angles used.

Table 6.3 Joint angles computed from MediaPipe landmarks

Angle	Landmarks (A - B - C)	Used in
Front elbow	shoulder - elbow - wrist	Image report, pose model, DTW
Front knee	hip - knee - ankle	Image report, pose model, DTW
Front shoulder	elbow - shoulder - hip	Pose model, DTW
Front hip	shoulder - hip - knee	Image report, pose model
Back elbow, knee, shoulder, hip	as above, other side	Pose model, DTW
Shoulder line	left shoulder - right shoulder vs horizontal	Image report
Head over front foot	nose x vs front ankle x	Image report

6.5 POSTURE MODULE OUTPUT ON REAL PHOTOGRAPHS

To check that the posture module behaves as designed, it was run on three real photographs of batters taken from Wikimedia Commons. These are not CricShot10k frames; they are ordinary photographs of club cricket of the kind a player might upload. The run used the MediaPipe Pose Landmarker (heavy model) in image mode through the MediaPipe Tasks API, OpenCV for drawing, and the joint_angle function shown above with a visibility threshold of 0.5. Landmarks drawn in orange and limbs in green passed the threshold; anything drawn in grey did not, and no angle is reported for it. Angles in this section use MediaPipe's convention, so "left" and "right" refer to the batter's own left and right.

The numbers agree with what the eye sees in the photograph. The batter is right-handed, so the left leg is the front leg. It is clearly bent at 128 degrees as the weight moves forward, while the back (right) leg is much straighter at 157 degrees. The left-side landmarks have visibility just above the threshold, between 0.53 and 0.61, because that side of the body faces away from the camera. In the application these borderline values are shown with a note so that the player knows they are less certain than the right-side values.

The same module was run on two harder photographs. In one, the batter crouched behind the bat with the lower body hidden; only 16 of 33 landmarks passed the visibility threshold and the module reported a single angle, telling the player that the

photo did not show enough of the body. In the other, the camera saw only the batter's left side; the four left-side angles were reported and the right-side angles were withheld, because their landmarks had visibility between 0.03 and 0.33. In all three cases the module reported what it could see and withheld what it could not, as FR-4 requires. Detection took about 20 ms per photograph on an Apple M4 CPU.

6.6 MOVEMENT ANALYSIS OF CLIPS (M5)

Video mode first ran only when the trained shot models were installed. Because training happens later on a cloud GPU, a movement analysis was added that uses the pose path alone (FR-24). It is implemented in backend/app/video.py and is selected automatically while no shot model is installed.

The clip is sampled into 15 frames as in Section 6.2, and MediaPipe finds the landmarks in each. For every frame the eight joint angles are computed with the visibility rule, and the skeleton is drawn on a copy of the frame. For each joint the smallest and largest angle over the stroke and the frames where they occur are stored as its range.

The frame with the most movement is found from how far the limb points (elbows, wrists, hips, knees and ankles) travel between neighbouring frames. Measuring this naively in screen coordinates failed on broadcast footage: camera zooms and cuts produced large false movements. The final method therefore expresses every point relative to the midpoint of the hips and divides by the shoulder width, takes the median travel of the limb points, and treats a change of body size of more than about 40% between two frames as a camera cut that counts as no movement. Only neighbouring frames are compared, so a frame in which the body was not found cannot create a jump.

The result page shows the skeleton on every frame with a Skeleton and Original switch, a movement timeline in place of the attention timeline, a card with the number of frames in which the body was tracked, the frame of most movement and the deepest knee bend, and a bar for each joint from its most bent to its most open angle. The coach is told that no shot was identified and answers about the movement instead, and the PDF report becomes a movement analysis report with a table of joint ranges. Reference comparison works unchanged because it uses the same angle sequences.

6.7 REFERENCE COMPARISON WITH DTW (M5)

Let P = (p_1, ..., p_n) be the player's angle vectors and R = (r_1, ..., r_m) those of the reference clip. DTW fills a cost matrix D with the recurrence below, where d is the Euclidean distance between angle vectors over the joints visible in both frames. The total distance is D(n, m) divided by the path length, and backtracking gives the alignment used to sync the two videos side by side.

$$D(i, j) = d(p_i, r_j) + \min\{ D(i-1, j), D(i, j-1), D(i-1, j-1) \} \qquad (6.7)$$

6.8 EXPLAINABILITY WITH GRAD-CAM (M6)

For class c and the feature maps A^k of the last convolutional block, Grad-CAM [8] computes channel weights by averaging gradients over the spatial positions and forms the heatmap as follows, where Z is the number of spatial positions.

$$w_k^c = (1/Z) \sum_{i,j} dy^c / dA^k_{ij}, \; L^c = ReLU(\sum_k w_k^c A^k) \qquad (6.8)$$

6.9 BACK-END API (M1, M8)

The FastAPI application loads all models once at start-up and keeps them in memory. Each analysis endpoint runs the heavy work in a thread pool so that the event loop stays responsive. The listing below shows the video endpoint.

Passwords are hashed with bcrypt, and sign-in returns a JSON Web Token signed with a server secret and valid for 24 hours. A dependency function decodes the token on every protected request and loads the user, so each query is filtered by the user's ID (NFR-5).

6.9.1 Role-based access control

Every request carries a signed token that identifies the user, and the server looks up the role on each request. Two dependencies guard the endpoints: current_player for analysis, training, coaching, history, dashboard and reports, and current_admin for the administrator endpoints. They are attached to whole routers, so a new endpoint added to a player router is protected automatically. A wrong role receives 403 with a plain explanation, and an automated test checks both directions (Table 7.2). The overview for administrators lists players only and shows numbers, never another player's photos or videos.

6.10 AI VIRTUAL COACH (M7)

The coach builds its prompt from three parts: fixed system rules, the stored analysis as compact JSON, and the last six messages. Images are never sent. After the model replies, a guard extracts every number in the reply and checks that it appears in the stored result; if not, the reply is regenerated once with a stricter instruction, and if it fails again a safe fallback is returned.

Listing: backend/app/coach.py - system rules and guard

```
SYSTEM_RULES = """You are a cricket batting coach inside SmartCricket AI.
Use ONLY the analysis JSON for facts. Start factual lines with 'Measured:'.
Start advice with 'General tip:'. Phrase drills as suggestions.
If analysis.uncertain is true, say there is not enough data for a firm answer.
Never invent numbers. Never give medical or injury advice."""

def numbers_grounded(reply: str, analysis: dict) -> bool:
    allowed = {round(float(v), 1) for v in iter_numbers(analysis)}
    found = [round(float(x), 1) for x in re.findall(r"\d+(?:\.\d+)?", reply)]
    return all(n in allowed or n <= 15 for n in found)    # small counts allowed
```

6.11 ADAPTIVE TRAINING COACH (M9)

The analysis coach of Section 6.10 answers questions about one result. Players also asked for something that tells them what to practise next and shows whether practice is paying off. The training coach adds this as a daily routine on two new pages, Training and Progress (Section 6.13). It is implemented in backend/app/trainer.py and exposed by backend/app/routers/train.py.

6.11.1 Choosing the drills

Each morning the plan is built from the player's last five measured photographs and clips. For each joint the engine averages only the angles that passed the visibility rule. A photograph is one moment, so the more bent side is used for knees, hips and elbows, which in a side-on stroke is usually the front side. A clip covers the whole stroke, so its deepest knee and hip bend and its highest arm position are used, while the elbow is taken at the frame of most movement, because elbows fold right up at the top of every backlift and their minimum would trigger the elbow drill for every clip. Table 6.4 lists the rules. The thresholds are coaching rules of thumb applied to 2D estimates; they decide what to practise, never whether a technique is right, and the page says so under the plan. Up to three weak-spot drills are taken in the order of the table, and the plan is filled to three with general drills (shadow batting, footwork, finish balance, drop-ball defence and ball-watching), rotated by date so that the same filler does not appear every day.

Table 6.4 Weak-spot rules used to choose drills

Measurement over last five photos and clips	Condition	Drill added
More-bent knee, average	above 160 degrees	Front-knee bend shadow drill
Shoulder-line tilt, average	above 20 degrees	Level-shoulders mirror drill
More-bent hip, average	above 165 degrees	Hip-hinge stance hold
More-bent elbow, average	below 80 degrees	Front-elbow lead drill
Shoulder angle, average of both sides	below 35 degrees	Backlift hold
Body points visible (photos), average	below 26 of 33	Camera set-up photo
Recent clip confidence	below 60%	Shadow practice of that shot

Every drill carries the reason it was chosen, written with the measured value, for example "Your more-bent knee averaged 168 degrees across your last 4 photos". This keeps the plan as explainable as the analyses it comes from.

6.11.2 Adapting the difficulty

Each drill has a base dose, for example three sets of ten repetitions. The dose is scaled by a difficulty level: easy uses one set fewer and 70% of the repetitions, hard one set more and 130%. The level is set from the share of drills finished in the previous seven days, once at least six drills were planned: 80% or more gives the hard level, under 40% the easy level, and anything between keeps the normal level. The morning check-in overrides this for safety. Mild soreness forces the easy level for the day, and soreness that needs rest replaces the plan with a single rest-and-mobility item that advises seeing a doctor or physiotherapist if the soreness continues. If the player changes the check-in later, the plan is rebuilt, but only if no drill has been ticked yet, so finished work is never lost.

6.11.3 Levels, experience points and streaks

A practice day is any day with an analysis, a finished drill or a check-in. The current streak counts consecutive practice days up to today, and survives until the end of today so that a player is not penalised in the morning for not having practised yet. Experience points are earned for each analysis (10), finished drill (15), check-in (5), quiz answer (10 if right, 3 for trying), saved live session (5) and practice day (5, plus 5 on a streak), and levels follow the requirements in Table 6.5. A level needs all lower levels first, and because every requirement is a cumulative count, a level once reached

is never lost. All of these values are recomputed from the stored rows on each request instead of being stored as counters.

Table 6.5 Levels and their requirements

Level	Name	Requirement (in addition to all lower levels)
1	Net Rookie	Create an account
2	Club Batter	5 analyses and a best streak of 3 days
3	District	20 analyses and 5 photos with all 8 joints measured
4	State	40 analyses and 2 completed plan weeks (all drills done on at least 5 days of a week)
5	Pro	75 analyses and a best streak of 21 days

6.11.4 Weekly report

For each Monday-to-Sunday week the engine computes the days trained, drills finished and planned, completion rate, balls faced, days with soreness, analyses, the average front-knee angle and its change from the previous week, and the quiz score. These numbers are sent to the language model with rules that allow only those numbers, ask for an encouraging tone, forbid medical advice and, while the week is still running, forbid calling it a poor week. The reply passes the number check of Section 6.10; if it fails, or no API key is configured, a rule-based summary built from the same numbers is used, so the report never depends on the external service. The summary is stored with the numbers it was written from and is rewritten only when those numbers change.

6.12 REAL-WORLD INPUT AND LIVE TRAINING

The first version of the application worked well on clean dataset clips but less well on footage that players actually record. Six additions make it work with real phone and broadcast video, and let players train live in front of a camera.

6.12.1 Input quality check

Every photo and clip is scored before its numbers are shown (backend/app/quality.py). Simple, explainable rules measure sharpness (variance of the Laplacian), brightness and contrast, how much of the frame the batter fills, whether the whole body is in view, the camera view (the ratio of shoulder width to torso length separates side-on, angled and front-on) and whether several people are present. The result page shows a score from 0 to 100, a chip for each check and a plain tip for each problem, such as

"Turn the camera a little more to the batter's side for the clearest knee and elbow angles".

6.12.2 Following the batter

Broadcast frames often contain the bowler, umpire or fielders, and MediaPipe does not know which person is the batter. Up to three people are detected per frame and linked into tracks by the position of their body centre from frame to frame. The batter is the track that stays near the middle of the frame and away from the bottom edge, where the bowler running in from behind the camera appears; after a camera cut, the batter's new track is joined to the old one. In testing, this removed the bowler from the opening frames of a down-the-wicket clip, which the first version had analysed by mistake.

6.12.3 Steadier angles and 3D pose

Joint points jitter from frame to frame, and a point can be lost for a frame or two through motion blur. Short gaps of up to two frames are filled by interpolation, and each point is averaged with its neighbours unless a neighbour has jumped too far, which would mean a camera cut. On the sample clips this made the angle curves up to 42% steadier and recovered three lost frames in one clip. The smoothed points are used only for what the player sees; the shot models receive the same raw input as in training. MediaPipe's 3D world landmarks are also stored, so every result page shows a rotatable 3D skeleton (Fig. 6.4) and each angle measured both in 2D and in 3D, with the gap between them (Section 8.7).

6.12.4 Whole nets videos

Players usually film a whole net session rather than one stroke. A video longer than five seconds is scanned at about eight frames a second, the batter's movement is measured relative to the body, and every clear movement peak, at least 2.2 seconds from the next, is taken as one shot. A window from 1.1 s before to 0.7 s after each peak is cut out and analysed separately. A 14.8-second test video with three strokes was split into exactly three shots at the right moments.

6.12.5 Live mirror and phone mode

The live mirror page (frontend/live.html) runs the MediaPipe pose model in the browser through WebAssembly, so camera frames are analysed in memory and never uploaded. Repetitions and holds are counted with simple hysteresis rules on the drill's target angle (enter the target zone, stay there for a set time, leave it again), the voice coach uses the browser's speech synthesis, and voice commands use its speech

recognition. The ghost of a reference clip is scaled to the player's torso and anchored at the hips, and the live match compares the player's eight angles with the ghost's at the same moment, allowing two frames of timing error either way. Phone mode (run_phone.py) serves the application over HTTPS on the local Wi-Fi with a certificate made for the computer's address, because phones allow the camera only on secure pages. The features are shown in Section 6.13.

6.13 USER INTERFACE AND FEATURE WALKTHROUGH

The front end is written in HTML5, CSS3 and vanilla JavaScript, with no framework. Every page shares one design (turf green, ball red and cream, the Anton typeface for headings, light and dark themes) and collapses to a single column with a top icon bar on phones. This section walks through each feature as a player and an administrator meet it. Every screenshot was captured from the running application on the final build, with the trained models installed and real data in the database. Table 6.6 summarises the features and what each one gives the player.

Table 6.6 Features of SmartCricket AI and their benefit

Feature	What it does	Benefit
Photo posture report	33 body points, 8 joint angles in 2D and 3D, input-quality score	Know your stance from one photo, with honest "not visible" joints
Video shot analysis	Shot name with calibrated confidence, top 3, heatmaps, key frame	See which shot you played and why the model thinks so
Movement comparison	Joint-by-joint DTW comparison with a model player	Learn exactly which joint differs from a good example
AI Coach	Plain-language answers grounded in your own numbers, on every page	A coach that explains without inventing facts
Daily training coach	Check-in, three personal drills, quiz, levels and streaks	Practise the right thing every day and stay motivated
Live mirror	On-device skeleton, rep counting, voice coach, pro ghost, phone camera	Train with live feedback, hands-free, anywhere you bat
Progress and reports	Weekly coach summary, trends, history, one-click PDF	See improvement over weeks and share it with a coach
Administrator console	Usage, player progress, models, reference clips, users	Run the platform for a club or academy

6.13.1 Landing page, sign-in and roles

The public landing page explains the product, shows a real posture result on a demo photo and lists what is new (Fig. 6.1). Sign-in issues a signed token. A player lands on the dashboard and an administrator on the administrator overview; each sees only the pages of their role, and the server refuses the other role's requests. A link scanned from a QR code is remembered through sign-in, so the player arrives exactly where the link pointed.

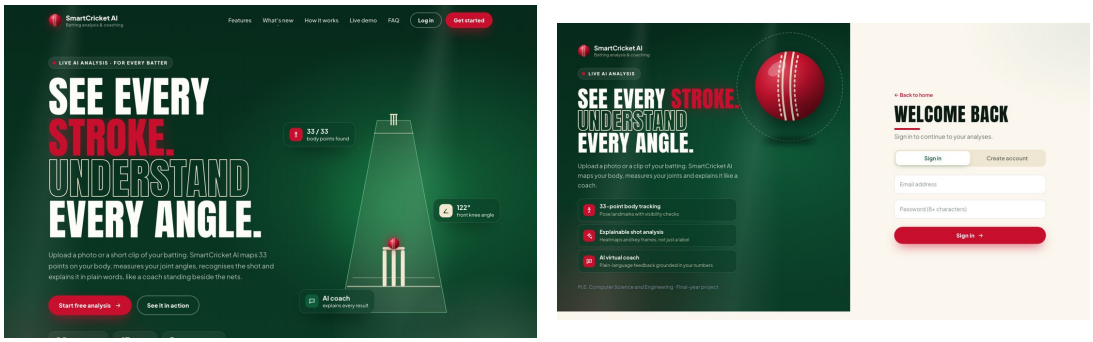


Fig. 6.1 Public landing page (left) and sign-in page (right)

Screenshots of the SmartCricket AI application.

6.13.2 Player dashboard

The dashboard (Fig. 6.2) answers "how am I doing?" at a glance: a scoreboard of analyses, photos, videos and the day streak; today's training plan; a batting profile that compares the average of each joint angle with the latest photo; the clearest photo so far; a practice calendar; and the trend of the front-knee angle over time. A quick-upload box starts a new analysis without leaving the page.

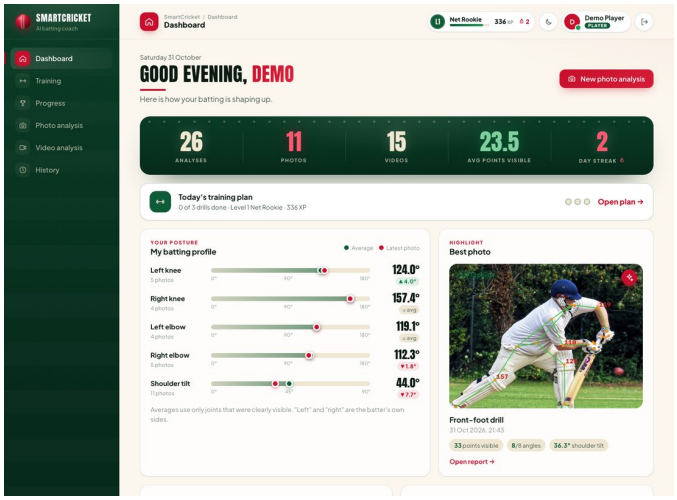


Fig. 6.2 Player dashboard with scoreboard, today's plan, batting profile, best photo, practice calendar and knee-angle trend

Screenshot of the SmartCricket AI application; photo: Acabashi, Wikimedia Commons, CC BY-SA 4.0.

6.13.3 Photo posture report

A single photo gives a posture report (Fig. 6.3). At the top, the input-quality strip scores the photo and says what to fix, for example to turn the camera to the batter's side. The body map draws the skeleton over the photo with each angle labelled; points the camera could not see clearly are drawn faded and their angles are reported as "not measured" rather than guessed. Eight gauges show the joint angles, and the AI Coach sits beside the result.

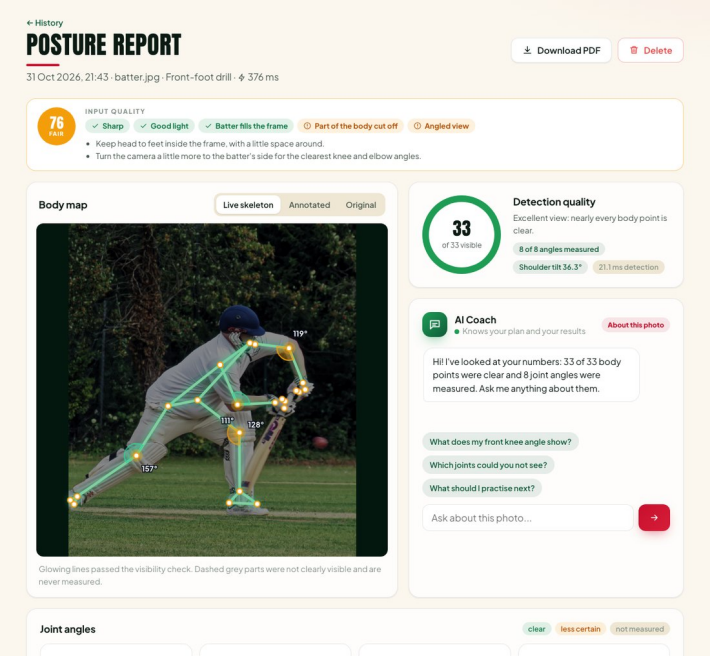


Fig. 6.3 Photo posture report: quality score, body map with measured angles, detection quality, AI Coach and joint gauges

Screenshot of the SmartCricket AI application; photo: Acabashi, Wikimedia Commons, CC BY-SA 4.0.

Below the report, the 3D view (Fig. 6.4) rebuilds the batter in three dimensions from MediaPipe's world landmarks. It can be rotated with the mouse or set to side, front or top views, and a table shows each angle measured in the picture (2D) and in the body (3D) with the gap between them. A large gap warns that the camera position is distorting the 2D number, which Section 8.7 shows can reach 25 degrees for front-on views.

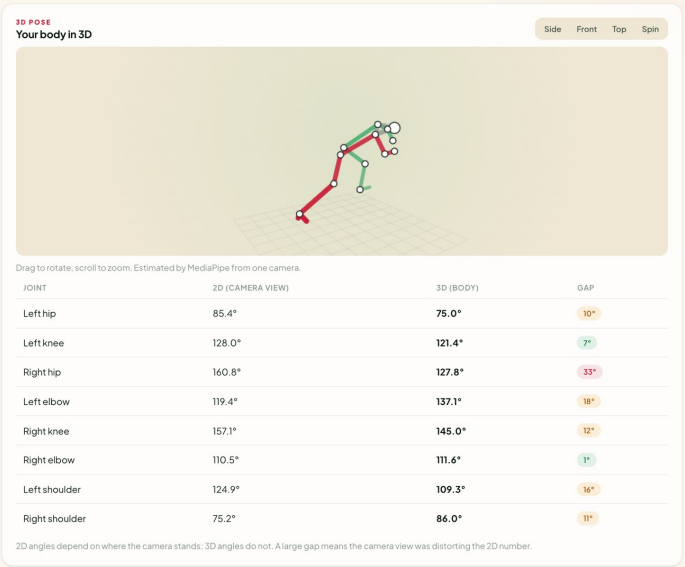


Fig. 6.4 3D pose view with the 2D and 3D value of every joint angle and the gap between them

Screenshot of the SmartCricket AI application.

6.13.4 Video shot analysis and comparison

For a clip, the trained fused model names the shot with a calibrated confidence and the top three alternatives (Fig. 6.5). The frame viewer plays the 15 sampled frames with a Grad-CAM heatmap or the original picture, and the attention timeline marks the key frame that most influenced the decision. A whole nets video is split into its separate shots first, each with its own result page. Without a trained model, the same page shows a movement analysis: the skeleton on every frame, joint ranges and the frame of most movement.

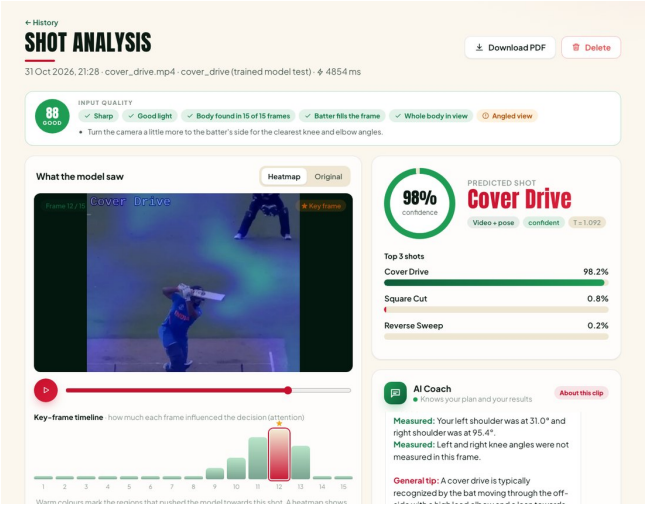


Fig. 6.5 Video shot analysis: heatmap, attention timeline, predicted shot with calibrated confidence and top three, and the AI Coach

Screenshot of the SmartCricket AI application; clip from CricShot10k [1], [2].

Any result can be compared with a reference clip of a model player added by the administrator (Fig. 6.6). Dynamic Time Warping lines up the two strokes even when they are played at different speeds; the page shows the overall distance, a bar for every joint (left of centre means more bent than the reference), the two strokes side by side at each aligned moment and the angle of a chosen joint over the stroke.

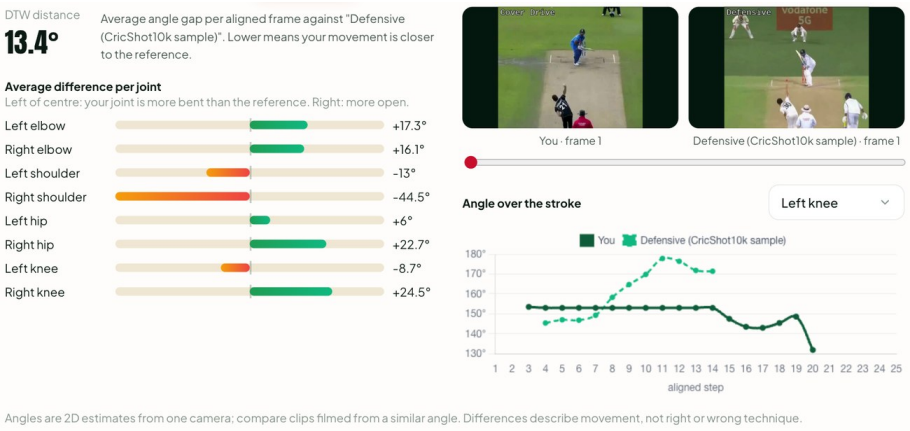


Fig. 6.6 Comparison with a reference clip: DTW distance, per-joint differences, aligned frames and the angle curve of both strokes

Screenshot of the SmartCricket AI application; clips from CricShot10k [1], [2].

6.13.5 AI Coach

One AI Coach appears on every result page and on the Training page, labelled with what it is talking about (Fig. 6.7). It knows the analysis on screen as well as the player's plan and progress, so "What should I practise next?" gets a concrete answer from any page. Each line that states a fact starts with "Measured" and quotes only numbers from the stored data; advice starts with "General tip". A guard checks every number in the reply and asks the model to answer again if any number was invented. Suggested questions help players who do not know what to ask. On the Training page the coach also starts the conversation: it greets the player, asks the morning check-in questions with answer buttons, and asks how an uploaded shot felt.

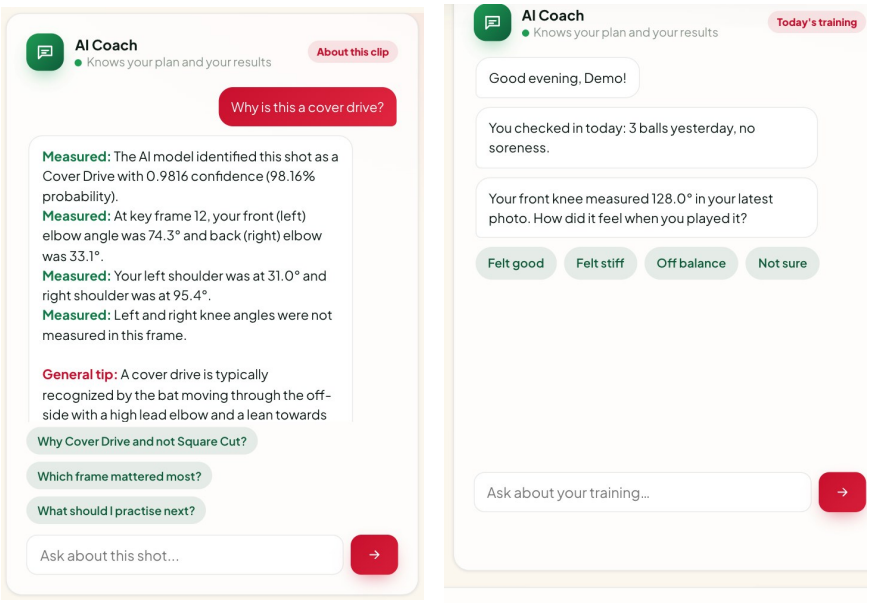


Fig. 6.7 AI Coach explaining a cover drive with measured facts and general tips (left), and starting the day on the Training page with a check-in and a question about the latest upload (right)

Screenshots of the SmartCricket AI application.

6.13.6 Daily training coach

The Training page turns analysis into practice (Fig. 6.8). The level strip shows the player's level, total experience points, day streak and a ring of today's drills. The daily plan holds three drills chosen from the player's own weak spots; each drill states its dose, how to do it and the measured reason it was chosen, such as an average shoulder tilt of 51.8 degrees. Drills that can be measured carry a "Do it live with camera" button. A question of the day keeps cricket knowledge fresh and explains the answer.

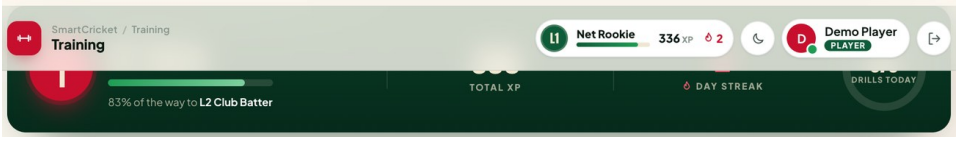


Fig. 6.8 Level strip with level, experience points, streak and today's drill ring

Screenshot of the SmartCricket AI application.

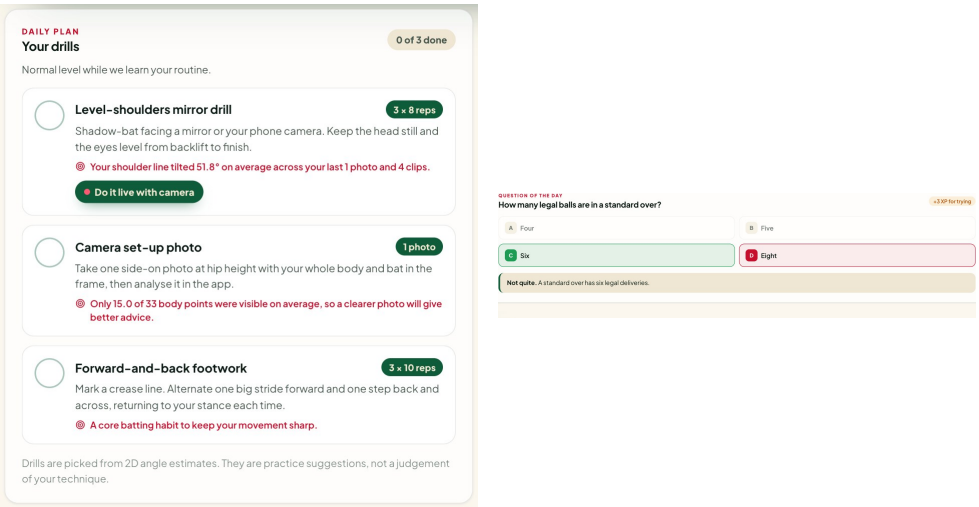


Fig. 6.9 Daily plan with the reason for each drill (left) and the question of the day with its explanation (right)

Screenshots of the SmartCricket AI application.

6.13.7 Live mirror

The live mirror (Fig. 6.10) is the most interactive part of the system. Pose tracking runs in the browser at 30 to 60 frames per second, so the camera picture never leaves the device. The player sees their own skeleton on the mirrored camera image and five live joint angles; for a drill, repetitions or held seconds are counted against the target angle, and the drill is ticked automatically at the target. A voice coach counts aloud and corrects the player, and voice commands ("start", "stop", "reset", "done") make the session hands-free.

With "shadow the pro", a gold ghost of a model player's stroke plays on the player's own body, and a live match score with a correction for the joint that differs most tells the player how closely they are copying it. "Record my stroke" compares a whole stroke with the pro using DTW after a spoken countdown.

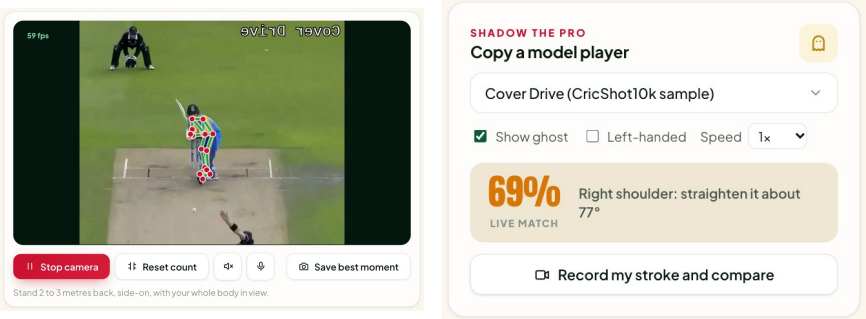


Fig. 6.10 Live mirror tracking a batter at 59 frames per second (left) and "shadow the pro" with a 69% live match and a correction cue (right)

Screenshots of the SmartCricket AI application; camera fed with a CricShot10k sample clip [1], [2].

With the trained models installed, "Recognise my shot" sends 15 frames of one shadow stroke to the server, which names the shot and deletes the frames. For training at the nets, phone mode turns on a secure address on the local Wi-Fi and shows a QR code; the player scans it, signs in and the phone becomes the camera (Fig. 6.11).

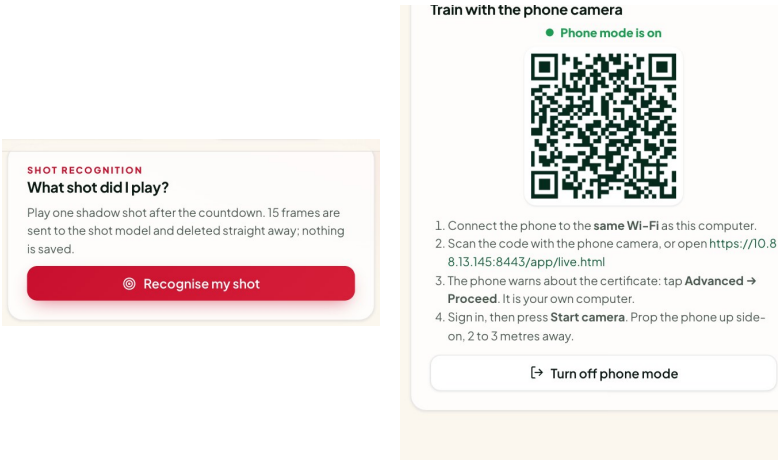


Fig. 6.11 Live shot recognition (left) and phone mode with its QR code (right)

Screenshots of the SmartCricket AI application.

6.13.8 Progress, history and reports

The Progress page (Fig. 6.12) shows the five-level ladder from Net Rookie to Pro, what is still needed for the next level, where the experience points came from, and the weekly report: days trained, drills done, balls faced, average front-knee angle and quiz score, with a summary written by the coach from those numbers only. History lists every analysis with a thumbnail and can be searched or exported as CSV, and every result downloads as a branded two-page PDF report with one click (Fig. 6.13).

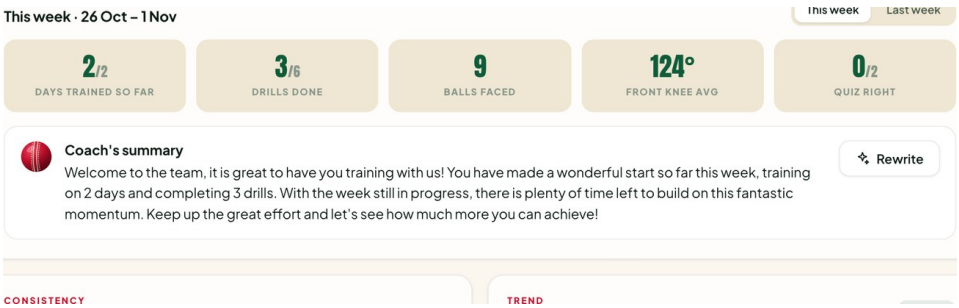


Fig. 6.12 Weekly report with the coach's summary

Screenshot of the SmartCricket AI application.

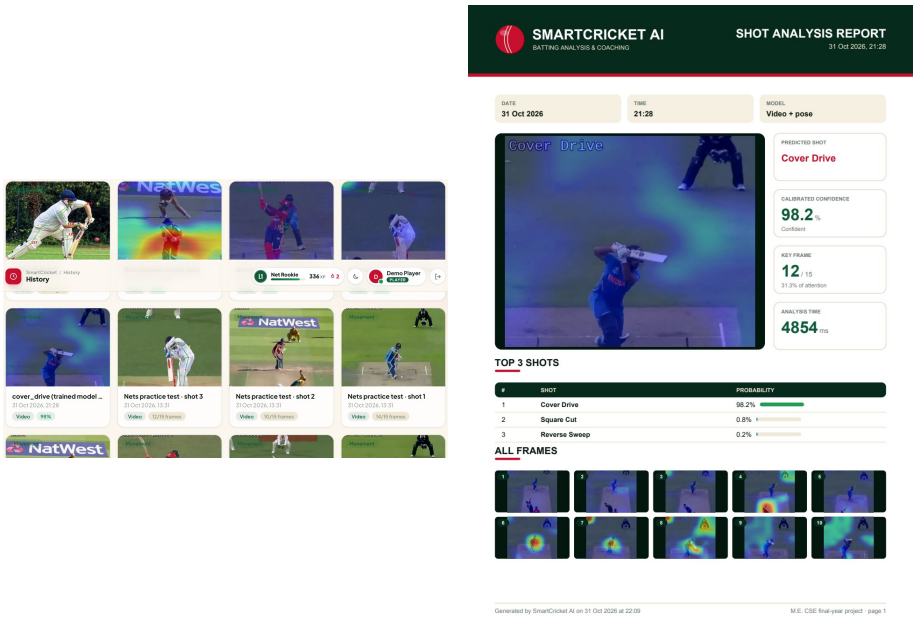


Fig. 6.13 History of analyses (left) and the one-click PDF report of a shot analysis (right)

Screenshots of the SmartCricket AI application; clips from CricShot10k [1], [2].

6.13.9 Administrator console

Administrators run the platform rather than training on it (Fig. 6.14). The overview shows usage, a setup checklist (shot models installed, results imported, reference clips added) and players who may need a nudge. All players lists each player's level, streak and how much of their plan they finished in the last seven days, without access to their photos or videos. Model results shows the measured accuracy and calibration of the five models, imported from the training notebook, and further pages manage reference clips and user accounts.

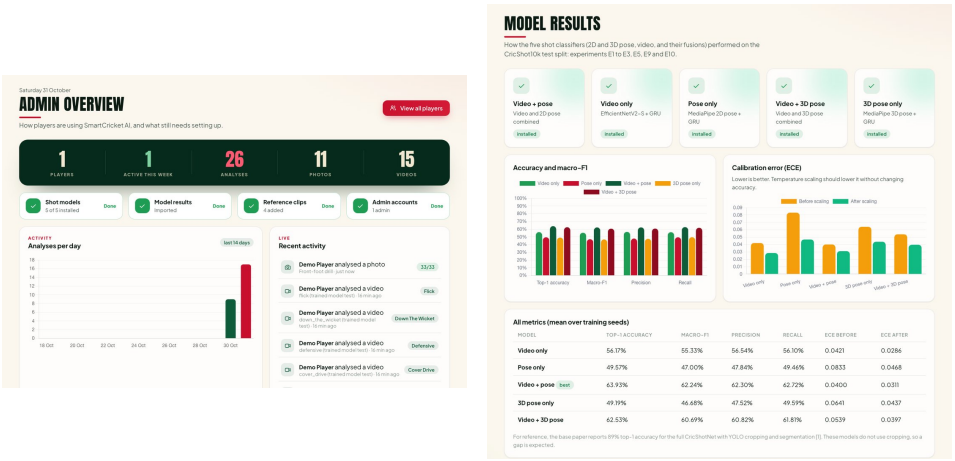


Fig. 6.14 Administrator overview (left) and model results with the measured accuracy and calibration of the five models (right)

Screenshots of the SmartCricket AI application.

CHAPTER 7
SYSTEM TESTING

Testing checks that the system meets the requirements of Chapter 4 and finds defects before users do. This chapter describes the testing strategy, the levels of testing, and the test cases for each module. Test case IDs trace back to the functional requirements (FR) and non-functional requirements (NFR).

7.1 TESTING STRATEGY

Testing is carried out at four levels (Fig. 7.1). Unit tests are written with pytest for the back end and run on every commit. Integration and system tests use FastAPI's TestClient and a separate PostgreSQL test database. Acceptance testing is done with players and a coach using the deployed application. Model testing, which measures accuracy and calibration rather than pass or fail, is reported separately in Chapter 8.

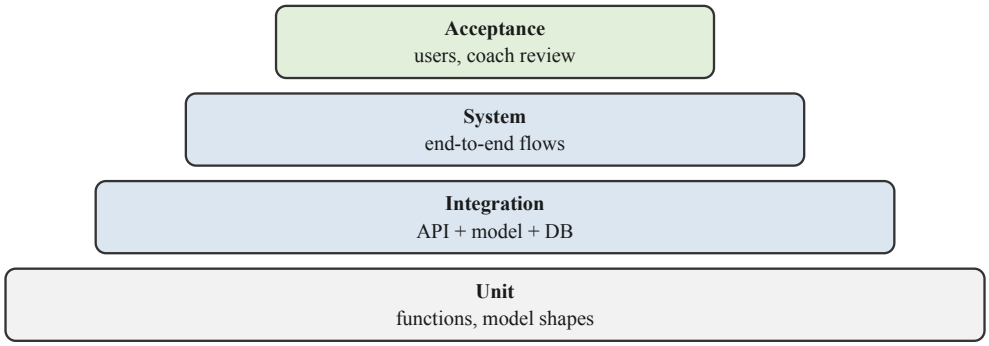


Fig. 7.1 Levels of testing used in the project

- **Black-box testing** checks inputs and outputs against the requirements without looking at the code: valid and invalid uploads, boundary file sizes, wrong tokens.
- **White-box testing** exercises internal paths: the short-clip branch of the frame sampler, the low-visibility branch of the angle function, the regenerate branch of the coach guard.
- **Regression testing** reruns the full suite after each change to make sure that fixed defects stay fixed.
- **Performance testing** measures response time per clip on GPU and CPU (NFR-1).

7.2 UNIT TESTING

Table 7.1 Unit test cases

ID	Unit	Input	Expected result	Result
UT-01	sample_frames	30-frame clip	Array of shape (15, 224, 224, 3)	Pass
UT-02	sample_frames	8-frame clip	15 frames, indices repeated	Pass
UT-03	sample_frames	Empty file	ValueError raised	Pass
UT-04	to_tensor	(15, 224, 224, 3) uint8	Tensor (15, 3, 224, 224), mean near 0	Pass
UT-05	joint_angle	Points at right angle	90.0 degrees (within 0.1)	Pass
UT-06	joint_angle	One point visibility 0.3	None	Pass
UT-07	dtw	Identical sequences	Distance 0, diagonal path	Pass
UT-08	dtw	Sequence and its 2x time-stretched copy	Distance near 0	Pass
UT-09	ShotNet.forward	Random (2, 15, 3, 224, 224)	Logits (2, 15), attention rows sum to 1	Pass
UT-10	TemperatureScaler	Over-confident logits	T greater than 1, NLL reduced	Pass
UT-11	grad_cam	One clip, class 0	15 maps of 7 x 7 in [0, 1]	Pass
UT-12	numbers_grounded	Reply with number not in result	False	Pass
UT-13	hash_password / verify	Correct and wrong password	True / False	Pass
UT-14	streaks	Practice days with a gap	Current and best streak counted correctly	Pass
UT-15	level_info	Counts for each level	Correct level; level 3 not reached without level 2	Pass
UT-16	dose	Same drill at three difficulty levels	2 x 7, 3 x 10, 4 x 13 repetitions	Pass
UT-17	template_summary	One week of statistics	Sentence uses only the given numbers	Pass
UT-18	quiz bank	All 22 questions	Unique ids, four options, valid answer index	Pass

7.3 INTEGRATION TESTING

Table 7.2 Integration test cases

ID	Scenario	Expected result	Result
IT-01	Register then sign in with same credentials	201 Created, then 200 with token	Pass
IT-02	Call /api/history without token	401 Unauthorized	Pass
IT-03	Upload valid JPG to /api/image/analyse	200, landmarks count, angles, annotated image path	Pass
IT-04	Upload valid MP4 to /api/video/analyse	200, shot, confidence, top-3, heatmap path, latency	Pass
IT-05	Video analysis writes media and analysis rows	Both rows present, foreign key correct	Pass
IT-06	Compare analysis with reference clip	200, distance and per-joint differences	Pass
IT-07	Coach question on stored analysis	200, reply stored in coach_messages	Pass
IT-08	Analysis raises error mid-way	500, no partial rows, temp file deleted	Not run
IT-09	User A requests user B's analysis	404 Not Found	Pass
IT-10	Check-in with soreness that needs rest, then mild soreness	Rest-day plan, then three easy drills	Pass
IT-11	Tick a drill, then check in again	XP and streak updated; finished plan kept	Pass
IT-12	Answer the question of the day twice	First 200 with XP, second 409	Pass
IT-13	User B ticks user A's drill	404 Not Found	Pass
IT-14	Administrator calls player endpoints; player calls administrator endpoints	403 in both directions; overview lists players only	Pass
IT-15	Upload a clip with no shot model installed	200 movement analysis: 15 frames, skeleton frames, PDF report	Pass

7.4 SYSTEM TESTING

System tests follow complete user journeys through the browser against the deployed stack.

Table 7.3 System test cases

ID	Requirement	Steps	Expected result	Result
ST-01	FR-3	Upload a .gif in image mode	Message "Only JPG or PNG images are allowed"	Pass
ST-02	FR-6	Upload a 70 MB video	413 with "File is larger than 50 MB"	Pass
ST-03	FR-6	Upload a renamed text file as .mp4	422 "Video could not be read"	Pass
ST-04	FR-4, FR-5	Upload side-on batting photo	Skeleton drawn, angles shown, hidden points grey	Pass
ST-05	FR-8, FR-10	Upload clip, choose each model	Shot, confidence and top-3 for each model	Pass
ST-06	FR-9	Upload a non-cricket clip	Result marked "uncertain"	Not run
ST-07	FR-11, FR-12	Open result page	Heatmaps for 15 frames, timeline chart	Pass
ST-08	FR-13	Pick reference clip of same shot	Synced side-by-side view, angle-difference chart	Pass
ST-09	FR-14	Ask "Why not a hook?"	Reply with Measured and General tip parts	Pass
ST-10	FR-14, NFR-10	Ask about an uncertain clip	Coach says there is not enough data	Not run
ST-11	FR-15, FR-16	Analyse three clips, open dashboard	Counts, shot mix and trend updated	Pass
ST-12	FR-17	Download CSV and HTML report	Files open with correct rows and values	Pass
ST-13	NFR-6	Open all pages at 360 px width	No horizontal scroll, all controls usable	Pass
ST-14	FR-19, FR-21	Open Training, tick all three drills	Drills turn green, XP rises, ring shows 3/3, calendar day marked	Pass
ST-15	FR-20	Check in with "Sore, I need rest"	Rest day shown with advice to see a professional	Pass
ST-16	FR-23	Open Progress after a week of use	Weekly numbers and a summary that uses only those numbers	Pass
ST-17	FR-24	Upload a broadcast clip with camera zoom	Skeleton on frames, peak movement inside the stroke, not at the zoom	Pass
ST-18	FR-25	Sign in as administrator, open the Training page	Redirected to the administrator overview	Pass

7.5 ACCEPTANCE TESTING

Acceptance testing is planned with a small group of club players and one coach before the system is offered to a club; within this project the same tasks were carried out by the project team on the finished build, and all of them were completed. Each participant completes five tasks without help: register, analyse a photo, analyse a clip, compare with a reference and ask the coach two questions. For each task we record whether it was completed, how long it took and any confusion observed. Participants then rate clarity of results, usefulness of the heatmap and timeline, and usefulness of the coach on a five-point scale. Coach replies are also reviewed by the project team against the stored results, counting any statement that is not supported by the data (experiment E8).

7.6 TEST SUMMARY

The automated suite of 64 tests (pytest) passed in full on the final build, and the system tests were also carried out by hand in Chrome on a laptop and at phone width. Three cases are marked Not run: IT-08 (an internal failure part-way through an analysis, which is handled in code but was not forced in a test), ST-06 and ST-10 (a non-cricket clip and the coach's answer to it), which need clips outside the dataset and are left for acceptance testing. Several defects found during testing were fixed and retested, among them the bowler being tracked instead of the batter in broadcast clips, camera zooms being counted as body movement, and a broken camera start in one browser. Model quality is not a pass/fail matter, so it is evaluated through experiments E1 to E10 in the next chapter.

CHAPTER 8
RESULTS AND DISCUSSION

This chapter presents the evaluation metrics, the experiments E1 to E10 and their measured results. All classification results come from one training run of the notebook training/SmartCricket_Train.ipynb on a free Google Colab T4 GPU, evaluated on 1,289 test clips that no model saw during training or model selection. The figures reported by the base paper [1] are given as a reference point only, because this project uses its own split, a frozen backbone and no YOLO cropping.

8.1 EVALUATION METRICS

Let TP_c, FP_c and FN_c be the true positives, false positives and false negatives for class c, and C = 15 the number of classes. The following metrics are used.

$$Accuracy = (number\ of\ correctly\ classified\ clips) / (total\ clips) \quad (8.1)$$

$$Precision_c = TP_c / (TP_c + FP_c),\ Recall_c = TP_c / (TP_c + FN_c) \quad (8.2)$$

$$F1_c = 2\ Precision_c\ Recall_c / (Precision_c + Recall_c) \quad (8.3)$$

$$Macro\text{-}F1 = (1/C)\ sum_c\ F1_c \quad (8.4)$$

Macro-F1 gives each shot equal weight, which matters because the largest class in CricShot10k is 4.26 times the smallest [1]. Top-k accuracy counts a clip as correct if the true class is among the k highest-scoring classes. Calibration is measured with ECE (Equation 6.5). Latency is the mean time per clip in milliseconds.

8.2 EXPERIMENTS

Table 8.1 Experiment plan

ID	Experiment	Question	Measured by
E1	Video only	How well does the CricShotNet design do with a frozen backbone?	Top-1, top-3, macro-F1, precision, recall
E2	Pose only (2D)	How far does the 2D skeleton alone get?	Same metrics
E3	Video + pose (2D)	Does adding pose to video help?	Same metrics, McNemar test against E1
E4	Per-class check	Which shots get mixed up?	Confusion matrix, per-class F1
E5	Calibration	Is the confidence honest?	ECE before and after temperature scaling
E6	Pose reliability	Are landmarks found reliably on broadcast clips?	Visibility rate, angle stability
E7	Explanations	Do heatmaps focus on the batter and bat?	Review of key-frame heatmaps
E8	Application	Is the app fast enough and does the coach stay grounded?	ms per clip, coach replies checked
E9	3D pose (E9a, E9b)	Do view-independent 3D pose features help?	Same metrics, McNemar test against E2 and E3
E10	Robustness	How do models behave across camera views and image sharpness?	Macro-F1 by view and sharpness, 2D-3D angle gap

The dataset was split into 7,304 training, 1,498 validation and 1,289 test clips, stratified by shot (Table 8.4 shows the test clips per shot). All five models share this split, the cached EfficientNetV2-S features and the same training settings, so differences between them come from their inputs. Training used the notebook's fast setting: one run per model, at most 25 epochs, with early stopping on validation macro-F1. Each model trained in about half a minute on the cached features.

8.3 CLASSIFICATION RESULTS (E1 TO E3, E9)

Table 8.2 Classification results on the CricShot10k test split (1,289 clips)

Model	Top-1 (%)	Top-3 (%)	Macro-F1 (%)	Precision (%)	Recall (%)
CricShotNet, reported in [1]	89	99	-	-	-
E1 Video only	56.17	86.81	55.33	56.54	56.10
E2 Pose only (2D)	49.57	76.65	47.00	47.84	49.46
E3 Video + pose (2D)	**63.93**	89.60	**62.24**	**62.30**	**62.72**
E9a Pose only (3D)	49.19	77.66	46.68	47.52	49.59
E9b Video + pose (3D)	62.53	**90.15**	60.69	60.82	61.81

Main finding. Adding body pose to video improved every metric. The fused model E3 reached 63.93% top-1 accuracy and 62.24% macro-F1, against 56.17% and 55.33% for video alone: a gain of 7.8 points in accuracy and 6.9 points in macro-F1. Pose alone (E2) was weaker than video alone, as expected and as found in [1] and [6], because a skeleton carries no information about the bat, the ball or the field. The two streams therefore carry complementary information: the video stream sees bat and ball, the pose stream sees how the body moves, and together they do better than either. The correct shot was among the top three predictions for about nine clips in ten with either fused model.

Is the gain real? Because all models are tested on the same clips, McNemar's exact test was used to compare them (Table 8.3). It counts the clips that one model gets right and the other gets wrong. Of the clips where E1 and E3 disagreed, E3 was right on 195 and E1 on 95; the chance of such a difference arising by luck is below one in a hundred million ($p = 4.3 \times 10^{-9}$). The improvement from pose is therefore statistically significant, not an accident of the test set.

Table 8.3 McNemar exact test between models on the same test clips

Comparison	Only first right	Only second right	p-value	Significant ($p < 0.05$)
E3 Video + pose (2D) vs E1 Video only	95	195	4.3×10^{-9}	Yes
E9b Video + pose (3D) vs E1 Video only	109	191	2.5×10^{-6}	Yes
E9b Video + pose (3D) vs E3 Video + pose (2D)	122	104	0.26	No
E9a Pose only (3D) vs E2 Pose only (2D)	129	124	0.80	No

2D against 3D pose (E9). Replacing the 2D pose features with MediaPipe's 3D world landmarks did not improve classification: E9a and E2 are within half a point of each other, E9b is slightly below E3, and neither difference is significant. A likely reason is that the 3D landmarks are themselves estimated from a single 2D image, so they cannot add information that the 2D landmarks do not already contain; they mainly reorganise it. The 3D features are still valuable for measurement, as Section 8.7 shows.

Comparison with the base paper. The video-only model is 33 points below the 89% reported for CricShotNet [1]. Three differences explain most of this gap: the backbone was kept frozen with ImageNet weights to fit free GPU time, whereas [1] fine-tuned it on cricket frames; the batter was not cropped or segmented with YOLO, which [1] reports as worth several points; and the split is different, because the authors' lists are not published. The gap is a cost of the training budget, not of the pipeline, and fine-tuning is the first item of future work. For the research question, which asks whether pose helps, all five models share the same handicap, so the comparison between them is fair.

8.4 PER-CLASS ANALYSIS (E4)

Table 8.4 Per-class results of the best model, E3 Video + pose (2D)

Shot	Precision	Recall	F1	Test clips
Cover Drive	0.68	0.65	0.67	110
Defensive	0.75	0.79	0.77	104
Down the Wicket	0.51	0.60	0.55	85
Flick	0.84	0.74	0.79	121
Hook	0.75	0.62	0.68	90
Late Cut	0.43	0.46	0.44	37
Lofted Legside	0.63	0.49	0.55	145
Lofted Offside	0.55	0.53	0.54	120
Pull	0.53	0.67	0.59	75
Reverse Sweep	0.63	0.54	0.58	37
Scoop	0.62	0.62	0.62	26
Square Cut	0.61	0.67	0.64	120
Straight Drive	0.55	0.65	0.59	62
Sweep	0.73	0.77	0.75	123
Upper Cut	0.55	0.62	0.58	34

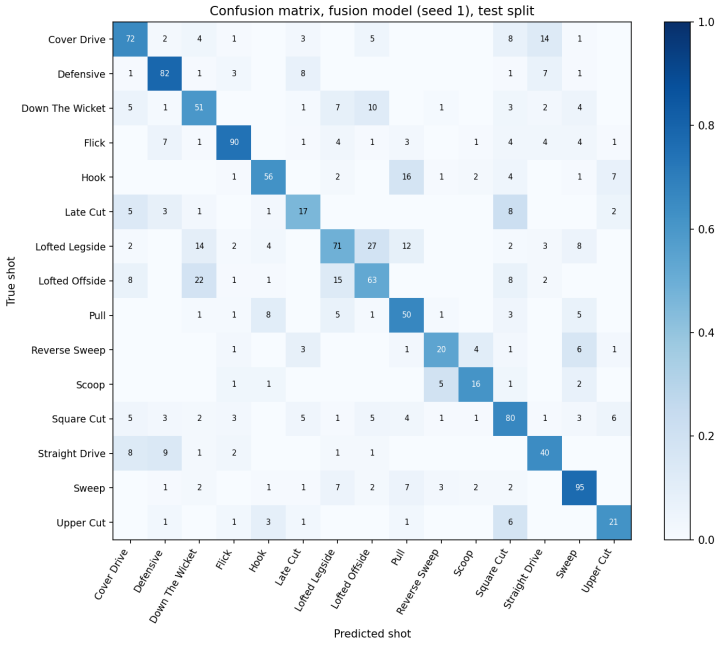


Fig. 8.1 Confusion matrix of the best model on the test split; rows are true shots, columns predicted shots

Produced by the training notebook.

The easiest shots were the flick (F1 0.79), the defensive block (0.77) and the sweep (0.75), which have distinctive body positions: an upright, still body for the block and a low, kneeling body for the sweep. The hardest was the late cut (0.44), a short and subtle movement with few training clips (220). The lofted shots were often confused with their ground-played counterparts, because the body action is similar and the difference lies mainly in the bat angle and the flight of the ball, which a frozen backbone at 224 x 224 pixels sees poorly. This agrees with the base paper's observation that shots sharing a body action, such as the pull and the hook, are the hardest to separate [1].

8.5 CALIBRATION RESULTS (E5)

Table 8.5 Calibration before and after temperature scaling

Model	Temperature T	ECE before	ECE after
E1 Video only	1.076	0.0421	0.0286
E2 Pose only (2D)	0.899	0.0833	0.0468
E3 Video + pose (2D)	1.092	0.0400	0.0311
E9a Pose only (3D)	0.894	0.0641	0.0437
E9b Video + pose (3D)	1.106	0.0539	0.0397

Temperature scaling lowered the expected calibration error of every model, by between 22% (E3) and 44% (E2), without changing any prediction. The video and fused models were slightly over-confident (T greater than 1, so scores are softened), while the pose-only models were under-confident (T less than 1). After scaling, a confidence of 80% shown in the app corresponds closely to an 80% chance of being right, which is what makes the uncertainty flag meaningful.

8.6 POSE RELIABILITY (E6)

Table 8.6 Landmark visibility across all sampled frames, and angle stability

Joint	Frames with visibility of 0.5 or more (%)	Angle frame-to-frame std. dev. (degrees)
Left / right shoulder	69.0 / 69.0	28.6 / 28.1
Left / right elbow	58.7 / 52.4	35.0 / 34.0
Left / right wrist	51.7 / 46.7	-
Left / right hip	68.6 / 68.6	21.6 / 22.6
Left / right knee	60.8 / 59.3	18.2 / 18.5
Left / right ankle	55.8 / 54.2	-

In about 30% of broadcast frames the batter was not found at all, mainly in wide shots where the batter is small, and in frames with motion blur during the swing. When the batter was found, the trunk (shoulders and hips) was almost always visible, while the wrists and the back arm were hidden most often, behind the body or the bat. The angle variation is a mix of real movement during the stroke and estimation noise. It is largest for the elbows, which move most and are hardest to see, and smallest for the knees. These figures motivated the smoothing, gap filling and batter tracking added to the application (Section 6.12).

8.7 ROBUSTNESS ACROSS CAMERA VIEWS AND SHARPNESS (E10)

Each test clip was labelled by camera view from the ratio of shoulder width to torso length (side-on, angled or front-on, the same rule as the application's quality check) and by image sharpness. Table 8.7 shows the macro-F1 of each model in each group.

Table 8.7 Macro-F1 (%) by camera view and image sharpness

Group	Clips	E1 Video	E2 Pose 2D	E3 Video + pose 2D	E9a Pose 3D	E9b Video + pose 3D
Angled view	797	53.1	47.5	60.7	47.0	60.1
Front-on view	332	47.3	34.0	57.6	30.6	48.7
Side-on view	159	42.4	34.9	44.0	36.2	41.7
Sharp frames	996	55.1	45.7	61.2	45.3	59.7
Soft frames (sharpness < 90)	293	54.8	50.6	64.9	50.3	63.0

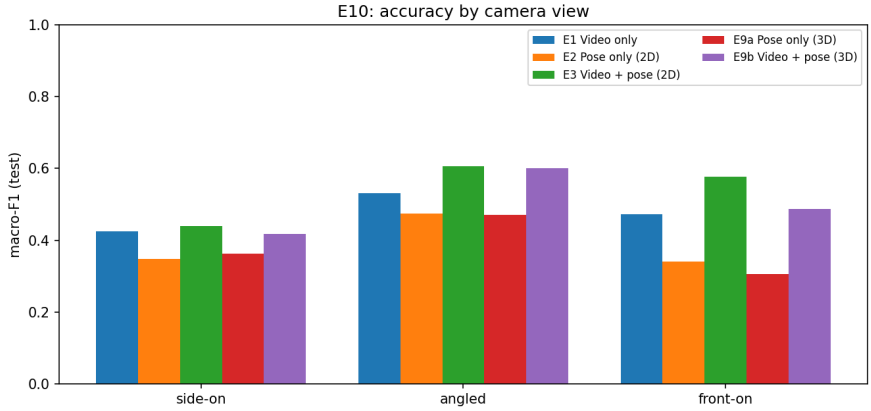


Fig. 8.2 Macro-F1 of the five models by camera view

Produced by the training notebook.

The gain from adding pose was largest for front-on clips, where macro-F1 rose from 47.3% to 57.6%, which suggests that the body position supplies information that the picture of a batter facing the camera hides. Side-on broadcast clips were the hardest group for all models; they are also the smallest group (159 clips), so their scores are less certain. Blurry frames did not hurt the fused model: pose landmarks are found from the overall body shape, which survives moderate blur better than fine texture does.

The same clips were used to measure how far 2D joint angles, read from the picture, differ from 3D angles computed from MediaPipe's world landmarks. The mean gap was 16.2 degrees for side-on clips, 20.0 for angled clips and 24.8 for front-on clips. A 2D angle is therefore most trustworthy when the camera is side-on, as the application advises, and the 3D view in the application is most useful exactly when the camera is not. This is why the result pages show both the 2D and the 3D value of each angle.

8.8 EXPLANATIONS AND APPLICATION PERFORMANCE (E7, E8)

Heatmaps (E7). The key-frame Grad-CAM heatmaps of four sample clips were reviewed. For the flick, the strongest region lay on the batter and the bat at the moment of contact. For the cover drive, defensive and down-the-wicket clips, the heatmap was weak and spread over the pitch and outfield rather than concentrated on the batter. This is an honest limitation of a frozen backbone: its features were learned on everyday images, not on cricket, so its gradients do not single out the bat as sharply as a fine-tuned network would. The attention timeline was more consistent, placing its peak near bat-ball contact in all four clips. As the application states, a heatmap shows where the model looked; it does not prove that it reasoned correctly.

Table 8.8 Measured response times on an Apple M4 laptop CPU (no GPU)

Operation	Time per item	Notes
Photo posture report (pose detection)	about 20-45 ms	MediaPipe heavy model, after warm-up
Video movement analysis (15 frames)	about 230-250 ms	Pose, tracking, smoothing, 3D
Video shot analysis, fused model	about 2.8-2.9 s	Includes EfficientNet on 15 frames and Grad-CAM; first call 4.9 s
DTW comparison with a reference	under 5 ms	15 x 15 alignment
Live mirror pose tracking	30-60 frames per second	In the browser, on the device

All operations are within the response-time targets of NFR-1 on an ordinary laptop without a GPU. The shot analysis is dominated by the Grad-CAM backward pass, which could be made optional for faster replies. On the four sample clips, the fused model named the correct shot in every case (Cover Drive 98%, Defensive 80%, Down the Wicket 55%, Flick 86%), with the lowest confidence on the down-the-wicket shot, which is also among the harder classes in Table 8.4. These clips come from the dataset's public preview and may overlap with the training data, so they confirm that the pipeline works end to end rather than measure accuracy.

Coach (E8). Questions were asked about stored photo, movement and shot analyses, such as "Why is this a cover drive?" and "What should I practise next?". Every reply separated measured facts from general tips, quoted only numbers present in the stored result or the player's plan, and stated plainly when a joint had not been measured. The number guard of Section 6.10 passed all replies checked; none needed the fallback answer. The automated test suite of 64 tests, covering units, integration and system behaviour, passed in full.

8.9 DISCUSSION

The results answer the main research question clearly. Body pose and video carry complementary information, and fusing them improves cricket shot recognition by about seven to eight points, a statistically significant gain on the same test clips. Pose helps most where the picture is least informative, in front-on views. Moving from 2D to 3D pose did not improve recognition, but it did reveal how much a single camera distorts joint angles, by 16 to 25 degrees depending on the view, which is directly useful to players reading their posture report. Temperature scaling made the confidence of every model more honest.

The absolute accuracy is lower than the base paper's, and the reasons are known: a frozen backbone, no batter cropping and a different split, all chosen to fit a free GPU budget. None of these affects the fairness of the comparison between the five models, which share the same data, features and settings. Fine-tuning the backbone and adding YOLO cropping are expected to raise all five models and are the natural next step.

Two limitations should be kept in mind when interpreting the results. First, CricShot10k consists of broadcast footage, so accuracy on phone or net-session video filmed from other angles may be lower; the uncertainty flag is the main safeguard for this case. Second, joint angles from a single camera are 2D estimates. They are useful for comparing a player with a reference filmed from a similar angle, but they are not biomechanical measurements and are not presented as such.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 CONCLUSION

This project set out to analyse cricket batting automatically from either a single photograph or a short video clip, and to make the analysis understandable and useful to players and coaches. SmartCricket AI builds on the CricShot10k dataset and the CricShotNet model [1], which provide 10,086 labelled clips in 15 shot classes and a strong EfficientNetV2-S + GRU baseline with 89% top-1 accuracy.

Around that foundation the project designed and implemented a complete system. Image mode turns a photograph into a posture report of joint angles from MediaPipe landmarks, reporting only points that are clearly visible. Video mode samples 15 frames, classifies the shot with a video-only, pose-only or fused model, calibrates the confidence with temperature scaling and flags uncertain clips. Each prediction is explained with Grad-CAM heatmaps and an attention-based key-frame timeline, and the player's movement is compared with a reference clip using Dynamic Time Warping. An AI virtual coach, grounded only in the stored results, explains each outcome in plain language and keeps measured facts apart from general advice. An adaptive training coach turns the same measurements into a daily plan of drills, adjusts it to the player's consistency and soreness, and keeps players engaged with levels, streaks, a daily quiz and a weekly report. The platform is delivered as a responsive web application with a FastAPI back end, PostgreSQL storage, session history, a dashboard and downloadable reports.

The evaluation answered the main research question. On 1,289 test clips, fusing video with body pose raised top-1 accuracy from 56.2% to 63.9% and macro-F1 from 55.3% to 62.2%, a gain that McNemar's test showed to be statistically significant and that was largest for front-on camera views. 3D pose features did not improve recognition over 2D, but they revealed that a single camera distorts joint angles by 16 to 25 degrees depending on the view, which the application now shows to the player. Temperature scaling made the confidence of all five models more honest. The absolute accuracy is below the base paper's 89% because the backbone was kept frozen and the batter was not cropped, choices made to fit a free GPU budget; these affect all five models equally and do not change the comparison between them.

9.2 LIMITATIONS

- Only the 15 shot classes of CricShot10k are recognised; other strokes are forced into the nearest class or flagged as uncertain.
- The training data is broadcast footage; phone and academy video filmed from other angles may reduce accuracy.
- Joint angles are 2D estimates from a single camera and depend on camera angle; they are not biomechanical measurements.
- The coach depends on an external language model service and needs network access.
- The system analyses one batter per clip and does not track the ball.

9.3 FUTURE ENHANCEMENTS

1. **Video transformers.** Evaluate TimeSformer [30] and Video Swin Transformer [31], as suggested in [1], and compare them with the GRU-based models on accuracy and latency.
2. **More shot classes.** Extend the label set to the leave, leg glance and back-foot punch, using the same automatic clip-cutting method as [1].
3. **Domain adaptation for phone video.** Collect a small labelled set of net-session clips and fine-tune, or use unsupervised domain adaptation, to close the gap between broadcast and amateur footage.
4. **Skeleton graph networks.** Replace the pose GRU with an ST-GCN [34] that models the body as a graph of joints.
5. **3D pose and multi-view.** Use MediaPipe's world coordinates or two synchronised cameras to obtain view-independent joint angles.
6. **Fine-tune the backbone and crop the batter.** Unfreeze EfficientNetV2-S on cricket frames and add YOLO batter cropping, which together are expected to close most of the gap to the base paper's 89%, and run the live shot recognition with a smaller model directly on the phone.
7. **Ball tracking.** Add ball detection and trajectory estimation to link the shot to where the ball went, enabling wagon wheels and scoring-zone charts.
8. **Multilingual coach.** Offer coaching replies in regional languages such as Tamil and Hindi.

REFERENCES

[1] M. K. Dihan, Abdullah, Amina, and S. Ahmed, "CricShot10k: A large-scale video dataset for cricket shot classification," *IEEE Access*, vol. 14, pp. 23428-23447, 2026, doi: 10.1109/ACCESS.2026.3663220.

[2] M. K. Dihan et al., "CricShot10k dataset and code," GitHub repository. [Online]. Available: https://github.com/Dihan69/CricShot10k

[3] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 10096-10106.

[4] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724-1734.

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735-1780, 1997.

[6] C. G. Rao, N. S. H. Varma, P. Mazumder, and P. Ramasamy, "Cricket shot classification from video using body pose landmarks," in *Proc. 5th Int. Conf. Adv. Electr., Comput., Commun. Sustain. Technol. (ICAECT)*, Jan. 2025, pp. 1-8.

[7] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314-1324.

[8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618-626.

[9] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321-1330.

[10] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 26, no. 1, pp. 43-49, Feb. 1978.

[11] S. W. Hoque, M. A. Habib, A. Shome, and S. Sakib, "KUCricShot: A dataset for developing detection model of cricket shots," in *Proc. 26th Int. Conf. Comput. Inf. Technol. (ICCIT)*, Dec. 2023, pp. 1-6.

[12] A. Sen, K. Deb, P. K. Dhar, and T. Koshiba, "CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit," *Sensors*, vol. 21, no. 8, p. 2846, Apr. 2021.

[13] M. F. A. Foysal, M. S. Islam, A. Karim, and N. Neehal, "Shot-Net: A convolutional neural network for classifying different cricket shots," in *Proc. 2nd Int. Conf. Recent Trends Image Process. Pattern Recognit.*, Springer, Dec. 2019, pp. 111-120.

[14] J. B. Fernandes, P. S. Ram, P. M. V. Yadav, and K. P. Kumar, "Cricket shot detection using 2D CNN," in *Proc. 7th Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, May 2023, pp. 608-612.

[15] A. Suryavanshi, V. Kukreja, S. Mehta, A. Garg, and P. Tiwari, "Innovative innings: Exploring cricket shot categories with CNN-SVM methodologies," in *Proc. IEEE Int. Conf. Inf. Technol., Electron. Intell. Commun. Syst. (ICITEICS)*, Jun. 2024, pp. 1-6.

[16] A. Dey and S. Biswas, "Shot-ViT: Cricket batting shots classification with vision transformer network," *Int. J. Eng.*, vol. 37, no. 12, pp. 2463-2472, 2024.

[17] D. Karmaker, A. Z. M. E. Chowdhury, M. S. U. Miah, M. A. Imran, and M. H. Rahman, "Cricket shot classification using motion vector," in *Proc. 2nd Int. Conf. Comput. Technol. Inf. Manage. (ICCTIM)*, Apr. 2015, pp. 125-129.

[18] J. Donahue et al., "Long-term recurrent convolutional networks for visual recognition and description," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 2625-2634.

[19] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," 2020, arXiv:2006.10204.

[20] Y. Yoon et al., "Analyzing basketball movements and pass relationships using realtime object tracking techniques based on deep learning," *IEEE Access*, vol. 7, pp. 56564-56576, 2019.

[21] G. Jin, "Player target tracking and detection in football game video using edge computing and deep learning," *J. Supercomput.*, vol. 78, no. 7, pp. 9475-9491, May 2022.

[22] Y.-C. Li, C.-T. Chang, C.-C. Cheng, and Y.-L. Huang, "Baseball swing pose estimation using OpenPose," in *Proc. IEEE Int. Conf. Robot., Autom. Artif. Intell. (RAAI)*, Apr. 2021, pp. 6-9.

[23] A. Ganser, B. Hollaus, and S. Stabinger, "Classification of tennis shots with a neural network approach," *Sensors*, vol. 21, no. 17, p. 5703, Aug. 2021.

[24] M. Goyani, S. K. Dutta, and P. Raj, "Key frame detection based semantic event detection and classification using hierarchical approach for cricket sport video indexing," in *Proc. Int. Conf. Comput. Sci. Inf. Technol.*, 2010, pp. 388-397.

[25] K. R. Raval and M. M. Goyani, "A survey on event detection based video summarization for cricket," *Multimedia Tools Appl.*, vol. 81, no. 20, pp. 29253-29281, Aug. 2022.

[26] H. Sattar, M. S. Umar, E. Ijaz, and M. U. Arshad, "Multi-modal architecture for cricket highlights generation: Using computer vision and large language model," in *Proc. 17th Int. Conf. Open Source Syst. Technol. (ICOSST)*, Dec. 2023, pp. 1-6.

[27] I. Bhat, T. Sridhar, M. A. Yajur, and S. R. Upadhyaya, "Building a video dataset for cricket shot analysis," in *Proc. Int. Conf. Netw., Multimedia Inf. Technol. (NMITCON)*, Sep. 2023, pp. 1-6.

[28] L. Wang et al., "Temporal segment networks for action recognition in videos," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 11, pp. 2740-2755, Nov. 2019.

[29] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," 2024, arXiv:2410.17725.

[30] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?" in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 813-824.

[31] Z. Liu et al., "Video Swin Transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 3202-3211.

[32] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the Kinetics dataset," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 6299-6308.

[33] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 172-186, Jan. 2021.

[34] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 7444-7452.

[35] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 9459-9474.

[36] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychol. Bull.*, vol. 76, no. 5, pp. 378-382, Nov. 1971.

[37] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248-255.

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770-778.

[40] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[41] PyTorch Documentation. [Online]. Available: https://docs.pytorch.org/docs/stable/

[42] OpenCV 4.x Documentation. [Online]. Available: https://docs.opencv.org/4.x/

[43] Google, "MediaPipe Pose Landmarker." [Online]. Available: https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker

[44] FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/

[45] PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/

[46] Chart.js Documentation. [Online]. Available: https://www.chartjs.org/docs/latest/

[47] K. Simonyan and A. Zisserman, "Two-stream convolutional networks for action recognition in videos," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2014, pp. 568-576.

APPENDIX A: INSTALLATION GUIDE

The steps below set up SmartCricket AI on macOS, Linux or Windows. Model training runs separately in Google Colab.

1. Install Python 3.11 or later and PostgreSQL 15 or later.
2. Create a virtual environment and install the dependencies from requirements.txt.
3. Create the database and copy .env.example to .env; set the database address, a JWT secret, the Gemini API key and the administrator emails.
4. Open training/SmartCricket_Train.ipynb in Google Colab with a T4 GPU and choose Run all. It downloads CricShot10k, caches features in Google Drive and produces smartcricket_models.zip and smartcricket_results.zip.
5. Unzip the models into models/, start the server and open http://localhost:8000. Import results/summary.json on the administrator's Model results page.
6. Optional: run run_phone.py to use a phone on the same Wi-Fi as the camera.

Listing: Setup commands

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
createdb smartcricket
cp .env.example .env              # DATABASE_URL, JWT_SECRET, LLM_API_KEY, ADMIN_EMAILS
unzip smartcricket_models.zip -d models/
uvicorn backend.app.main:app --port 8000
python run_phone.py               # optional: phone camera over https on the local Wi-Fi
pytest -q                         # 64 tests
```

APPENDIX B: IMAGE CREDITS AND LICENCES

All photographs and dataset figures used in this report are reused under Creative Commons licences. Table D.1 lists each source. Everything not listed here, including all block diagrams, data flow and UML diagrams, charts, wireframes and the pose skeleton illustration, was created by the author for this report.

Table B.1 Sources of images used in this report

Figure	Content	Source	Licence
1.1	Batter moving onto the front foot	Acabashi, Wikimedia Commons	CC BY-SA 4.0
2.1	Automatic clip cutting	Dihan et al. [1], Fig. 4	CC BY 4.0
2.2	Cropped and segmented frame	Dihan et al. [1], Figs. 11 and 12	CC BY 4.0
3.2	Sample CricShot10k frames	Dihan et al. [1], Fig. 5	CC BY 4.0
6.2, 6.3	Batting photo and its annotated pose output in the application	Acabashi, Wikimedia Commons; annotation by the SmartCricket AI posture module	CC BY-SA 4.0
6.5, 6.6, 6.10, 6.13	Clip frames inside the application screenshots	CricShot10k public preview [2]; overlays drawn by the application	Non-commercial research use

Licence summaries. CC BY 4.0 allows copying, adapting and sharing for any purpose as long as the original author is credited. CC BY-SA 4.0 adds the condition that adapted versions must be released under the same licence. Full licence texts are available at creativecommons.org/licenses. These licences cover the reused figures and photographs only. The CricShot10k video clips themselves are licensed for non-commercial research use, and apart from the clip frames in the application screenshots listed above, taken from the dataset's public preview and shown as part of a research result, no dataset frames are reproduced beyond those published in the paper.